



# OSR eHSM Bootloader 技术参考手册

|    |                |
|----|----------------|
| 版本 | 1.1            |
| 作者 | OSR IP         |
| 部门 | OSR IP         |
| 日期 | 2026 - 06 - 26 |
| 分类 | 用户手册           |

## Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2026 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

## 版权声明和专有信息声明

版权所有 © 2014 - 2026 深圳市纽创信安科技开发有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技开发有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技开发有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

## 修订历史

| 版本  | 修改日期       | 描述                                                                                                                                                                                                                                                                                                                                         |
|-----|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0 | 2025-01-01 | 初始版本                                                                                                                                                                                                                                                                                                                                       |
| 1.1 | 2026-06-25 | <ul style="list-style-type: none"><li>修改 图 2，增加 OTP Patch Cfg、Patch Addr、Patch Data 区域，eHSM/SOC 版本计数器偏移改为 0x030/0x040，OTP 密钥排布起始偏移更新为 0x118</li><li>新增 章节 3.5，说明硬件 OTP ROM Patch 机制和 OTP Patch 配置表编码</li><li>修改 表 9，OTP 密钥排布起始偏移更新为 0x118</li><li>新增 表 4，说明不同生命周期下 OTP Patch 配置表读写限制</li><li>明确软件 Patch 镜像章节不适用于硬件 OTP ROM Patch</li></ul> |

## 目录

|                               |    |
|-------------------------------|----|
| 1 概述 .....                    | 9  |
| 2 启动与错误处理 .....               | 10 |
| 2.1 启动流程 .....                | 10 |
| 2.2 错误处理 .....                | 11 |
| 3 OTP .....                   | 13 |
| 3.1 生命周期 .....                | 14 |
| 3.1.1 OTP Patch 配置表读写权限 ..... | 14 |
| 3.2 UID .....                 | 15 |
| 3.3 OTP 控制字段 .....            | 15 |
| 3.4 版本计数器 .....               | 17 |
| 3.5 Patch 机制 .....            | 17 |
| 3.5.1 硬件支持 .....              | 17 |
| 3.5.2 OTP Patch 配置表 .....     | 18 |
| 3.6 OTP 密钥排布 .....            | 19 |
| 3.7 OTP 驱动要求 .....            | 20 |
| 3.7.1 OTP 数据读取 .....          | 20 |
| 3.7.2 OTP 数据写入 .....          | 20 |
| 3.7.3 驱动 API .....            | 20 |
| 4 密钥烧录 .....                  | 21 |
| 5 鉴权 .....                    | 23 |
| 5.1 调试鉴权 .....                | 23 |
| 5.2 固件控制鉴权 .....              | 24 |
| 6 安全启动 .....                  | 25 |
| 6.1 安全启动交互流程 .....            | 25 |
| 6.1.1 并行启动模式 .....            | 25 |
| 6.2 部署说明 .....                | 26 |
| 6.2.1 eHSM FW 部署 .....        | 26 |
| 6.2.2 SOC FW 部署 .....         | 27 |
| 6.3 镜像格式 .....                | 28 |
| 6.4 验证命令定义 .....              | 29 |
| 6.5 验证流程 .....                | 29 |
| 6.6 配置说明 .....                | 30 |
| 6.6.1 算法配置 .....              | 30 |
| 6.6.2 存储访问方式配置 .....          | 31 |
| 7 安全固件升级 .....                | 32 |
| 7.1 升级镜像制作 .....              | 32 |
| 7.2 升级镜像安装 .....              | 33 |
| 7.2.1 算法配置 .....              | 34 |
| 8 补丁机制 .....                  | 35 |
| 8.1 硬件支持 .....                | 35 |
| 8.2 软件方案 .....                | 35 |

|        |                                |    |
|--------|--------------------------------|----|
| 8.2.1  | 补丁加载 .....                     | 36 |
| 8.2.2  | 补丁命中 .....                     | 36 |
| 8.2.3  | Patch 格式 .....                 | 37 |
| 8.2.4  | Patch 位置 .....                 | 38 |
| 8.2.5  | Patch 开发 .....                 | 38 |
| 9      | Mailbox 通信协议 .....             | 39 |
| 9.1    | 硬件说明 .....                     | 39 |
| 9.2    | 协议说明 .....                     | 39 |
| A      | Mailbox 命令 .....               | 41 |
| A.1    | 全局定义 .....                     | 41 |
| A.2    | BootLoader 功能 .....            | 41 |
| A.2.1  | bl_read_ver .....              | 42 |
| A.2.2  | bl_otp_read .....              | 42 |
| A.2.3  | bl_otp_write .....             | 43 |
| A.2.4  | bl_reg_rd .....                | 43 |
| A.2.5  | bl_reg_wr .....                | 44 |
| A.2.6  | bl_fw_upgrade .....            | 44 |
| A.2.7  | bl_verify_image .....          | 45 |
| A.2.8  | bl_get_random_key .....        | 46 |
| A.2.9  | bl_encrypt_key .....           | 46 |
| A.2.10 | bl_get_challenge .....         | 47 |
| A.2.11 | bl_debug_auth .....            | 48 |
| A.2.12 | bl_close_debug .....           | 48 |
| A.2.13 | bl_set_uart_baudrate .....     | 49 |
| A.2.14 | bl_self_test .....             | 49 |
| A.2.15 | bl_read_self_test_result ..... | 50 |
| A.2.16 | bl_fw_auth .....               | 51 |
| B      | 错误码 .....                      | 52 |
| C      | 字符调试鉴权协议 .....                 | 55 |
| C.1    | 获取挑战值指令 .....                  | 55 |
| C.2    | 签名校验指令 .....                   | 55 |
| C.3    | 关闭调试指令 .....                   | 56 |
| C.4    | 测试指令 .....                     | 56 |
|        | 参考文献 .....                     | 57 |

## 图目录

|      |                             |    |
|------|-----------------------------|----|
| 图 1  | Bootloader 启动流程 .....       | 10 |
| 图 2  | OTP 分布 .....                | 13 |
| 图 3  | CPU 读取指令流程 .....            | 19 |
| 图 4  | 生成密钥并烧录流程 .....             | 21 |
| 图 5  | 加密密钥并烧录流程 .....             | 22 |
| 图 6  | 调试鉴权流程 .....                | 23 |
| 图 7  | 并行启动流程 .....                | 25 |
| 图 8  | eHSM FW 基于 IRAM 的部署方式 ..... | 26 |
| 图 9  | eHSM FW 基于 NVM 的部署方式 .....  | 26 |
| 图 10 | SoC FW 基于 RAM 的部署方式 .....   | 27 |
| 图 11 | SoC FW 基于 NVM 的部署方式 .....   | 27 |
| 图 12 | 镜像格式示意图 .....               | 28 |
| 图 13 | 镜像的验证流程 .....               | 30 |
| 图 14 | 制作升级镜像 .....                | 32 |
| 图 15 | 软件补丁方案概览 .....              | 35 |
| 图 16 | 补丁加载流程 .....                | 36 |
| 图 17 | 补丁命中流程 .....                | 37 |
| 图 18 | Patch 加载位置示例 .....          | 38 |
| 图 19 | MailBox 协议数据流 .....         | 39 |
| 图 20 | Mailbox 协议流程 .....          | 40 |

## 表目录

|      |                                       |    |
|------|---------------------------------------|----|
| 表 1  | BL 错误处理定义 .....                       | 11 |
| 表 2  | 软件错误信号与 BL 行为 .....                   | 11 |
| 表 3  | 生命周期编码 .....                          | 14 |
| 表 4  | OTP Patch 配置表读写权限 .....               | 14 |
| 表 5  | FW Control-eHSM 字段说明 .....            | 15 |
| 表 6  | FW Control-SOC 字段说明 .....             | 16 |
| 表 7  | Version Counter 编码示例 .....            | 17 |
| 表 8  | OTP Patch 配置表 .....                   | 18 |
| 表 9  | OTP 密钥排布表 .....                       | 19 |
| 表 10 | 鉴权对应密钥 .....                          | 23 |
| 表 11 | 安全启动镜像字段说明 .....                      | 28 |
| 表 12 | 升级镜像字段说明 .....                        | 32 |
| 表 13 | 枚举 process_mode 定义 .....              | 41 |
| 表 14 | 枚举 debug_auth_challenge_type 定义 ..... | 41 |
| 表 15 | 结构体 ehsm_version 定义 .....             | 41 |
| 表 16 | 结构体 CMD-BL-READ-VER 定义 .....          | 42 |
| 表 17 | 结构体 RSP-BL-READ-VER 定义 .....          | 42 |
| 表 18 | 结构体 CMD-BL-OTP-READ 定义 .....          | 42 |
| 表 19 | 结构体 RSP-BL-OTP-READ 定义 .....          | 43 |
| 表 20 | 结构体 CMD-BL-OTP-WRITE 定义 .....         | 43 |
| 表 21 | 结构体 RSP-BL-OTP-WRITE 定义 .....         | 43 |
| 表 22 | 结构体 CMD-BL-REG-RD 定义 .....            | 43 |
| 表 23 | 结构体 RSP-BL-REG-RD 定义 .....            | 44 |
| 表 24 | 结构体 CMD-BL-REG-WR 定义 .....            | 44 |
| 表 25 | 结构体 RSP-BL-REG-WR 定义 .....            | 44 |
| 表 26 | 结构体 CMD-BL-FW-UPGRADE 定义 .....        | 44 |
| 表 27 | 结构体 RSP-BL-FW-UPGRADE 定义 .....        | 45 |
| 表 28 | 结构体 CMD-BL-VERIFY-IMAGE 定义 .....      | 45 |
| 表 29 | 结构体 RSP-BL-VERIFY-IMAGE 定义 .....      | 46 |
| 表 30 | 结构体 CMD-BL-GET-RANDOM-KEY 定义 .....    | 46 |
| 表 31 | 结构体 RSP-BL-GET-RANDOM-KEY 定义 .....    | 46 |
| 表 32 | 结构体 CMD-BL-ENCRYPT-KEY 定义 .....       | 46 |
| 表 33 | 结构体 RSP-BL-ENCRYPT-KEY 定义 .....       | 47 |
| 表 34 | 结构体 CMD-BL-GET-CHALLENGE 定义 .....     | 47 |
| 表 35 | 结构体 RSP-BL-GET-CHALLENGE 定义 .....     | 47 |
| 表 36 | 结构体 CMD-BL-DEBUG-AUTH 定义 .....        | 48 |
| 表 37 | 结构体 RSP-BL-DEBUG-AUTH 定义 .....        | 48 |
| 表 38 | 结构体 CMD-BL-CLOSE-DEBUG 定义 .....       | 48 |
| 表 39 | 结构体 RSP-BL-CLOSE-DEBUG 定义 .....       | 49 |
| 表 40 | 结构体 CMD-BL-SET-UART-BAUDRATE 定义 ..... | 49 |
| 表 41 | 结构体 RSP-BL-SET-UART-BAUDRATE 定义 ..... | 49 |

|                                                |    |
|------------------------------------------------|----|
| 表 42 结构体 CMD-BL-SELF-TEST 定义 .....             | 49 |
| 表 43 结构体 RSP-BL-SELF-TEST 定义 .....             | 50 |
| 表 44 结构体 CMD-BL-READ-SELF-TEST-RESULT 定义 ..... | 50 |
| 表 45 结构体 RSP-BL-READ-SELF-TEST-RESULT 定义 ..... | 50 |
| 表 46 结构体 CMD-BL-FW-AUTH 定义 .....               | 51 |
| 表 47 结构体 RSP-BL-FW-AUTH 定义 .....               | 51 |
| 表 48 错误码定义 .....                               | 52 |



# 1 概述

本文档描述 OSR eHSM BootLoader 相关的功能和技术要点。

## 2 启动与错误处理

### 2.1 启动流程

Bootloader 的启动流程如下图所示：

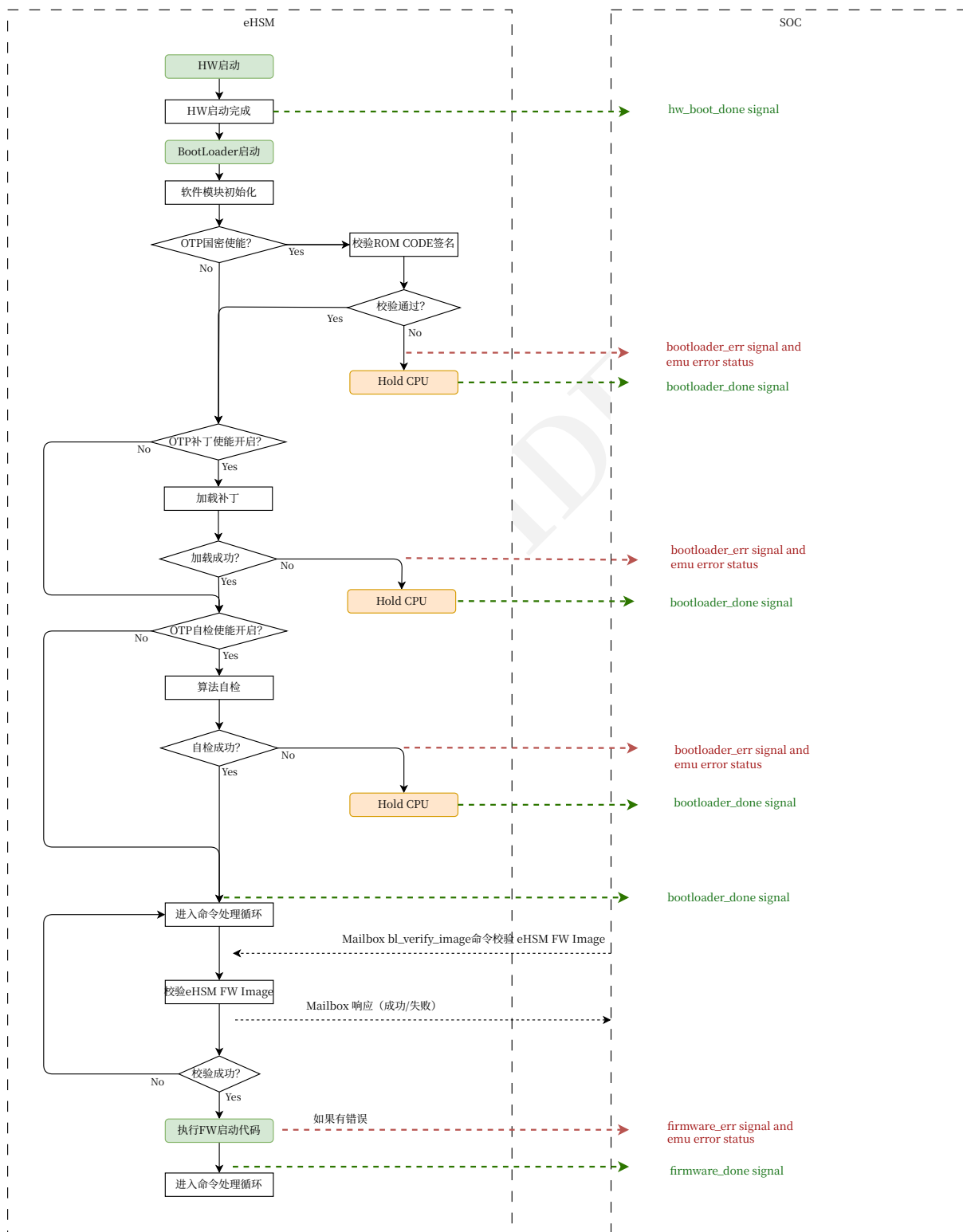


图1 Bootloader 启动流程

注：

- 启动流程中在模块初始化后会根据 OTP 中的配置决定是否开启 Watchdog 保护；
- 启动阶段的 `bootloader_done`<sup>1</sup> 表示启动完成，但并不一定能收发 Mailbox 命令，是否成功要再查询 `bootloader_err`<sup>2</sup> 信号；
- 硬件的启动流程请参考 [1] 的 5.2 Hardware Boot Flow

## 2.2 错误处理

下表中定义了 Bootloader 对错误处理的动作缩写：

表 1 BL 错误处理定义

| 代号 | 处理方式                                                                                                                                                                                                                                                                                                                                      |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | 忽略错误，继续运行                                                                                                                                                                                                                                                                                                                                 |
| 2  | <ul style="list-style-type: none"> <li>• 如果 BootLoader 处于启动阶段，则拉起 <code>bootloader_err</code> 和 <code>bootloader_done</code> 信号，然后 Hold 住 CPU 不再执行 (<code>while (1)</code>)</li> <li>• 如果 BootLoader 处于命令处理阶段（即 <code>bootloader_done</code> 已拉起），则 Hold 住 CPU 不再执行 (<code>while (1)</code>) 且不会拉起 <code>bootloader_err</code></li> </ul> |

Bootloader 软件运行过程中可能通过 EMU 主动汇报错误，下表中的错误 bits 对应 [1] 中 EMU `ERR_FW_0` 和 EMU `ERR_FW_1` 寄存器的 bits。

表 2 软件错误信号与 BL 行为

| SW Error Code            | ERR_FW bit | 说明                                    | BL 处理方式 |
|--------------------------|------------|---------------------------------------|---------|
| secboot_selftest_fail    | 0          | 自检时有算法失败                              | 2       |
| selftest_hash_fail       | 16         | HASH 自检失败                             | 2       |
| selftest_ske_fail        | 17         | SKE 自检失败                              | 2       |
| selftest_pke_fail        | 18         | PKE 自检失败                              | 2       |
| selftest_trng_fail       | 19         | 随机数自检失败                               | 2       |
| wdg_timeout              | 37         | Watchdog 超时，此位由软件设置                   | 2       |
| bl_verify_failed         | 40         | BootLoader ROM code 校验失败，ROM 可能已损坏    | 2       |
| cpu_exception            | 41         | CPU 异常                                | 2       |
| patch_load_failed        | 42         | 加载补丁失败                                | 2       |
| otp_init_failed          | 43         | 调用 <code>otp_init</code> 驱动初始化 OTP 失败 | 2       |
| fault_injection_detected | 44         | 检测到故障注入攻击                             | 2       |
| uid_crc32_mismatch       | 62         | OTP 中 UID 的 CRC32 值不正确，非 TEST 模式会检查   | 1       |
| hw_version_mismatch      | 63         | 软件与硬件的版本或客户编号不匹配                      | 1       |

注：

- 软件错误信号的低 48 位作为错误位，发生错误后软件会 Hold CPU 不再继续运行
- 软件错误信号的高 16 位作为警告位，发出警告后软件继续运行

<sup>1</sup>旧版文档此信号为 `fw_boot_done`

<sup>2</sup>旧版文档此信号为 `fw_boot_err`

- 软件信号是电平信号，软件不会清除对应信号
- 在生命周期 TEST MODE，软件会忽略所有错误，尝试继续运行

### 3 OTP

OTP 的数据分布如下图：

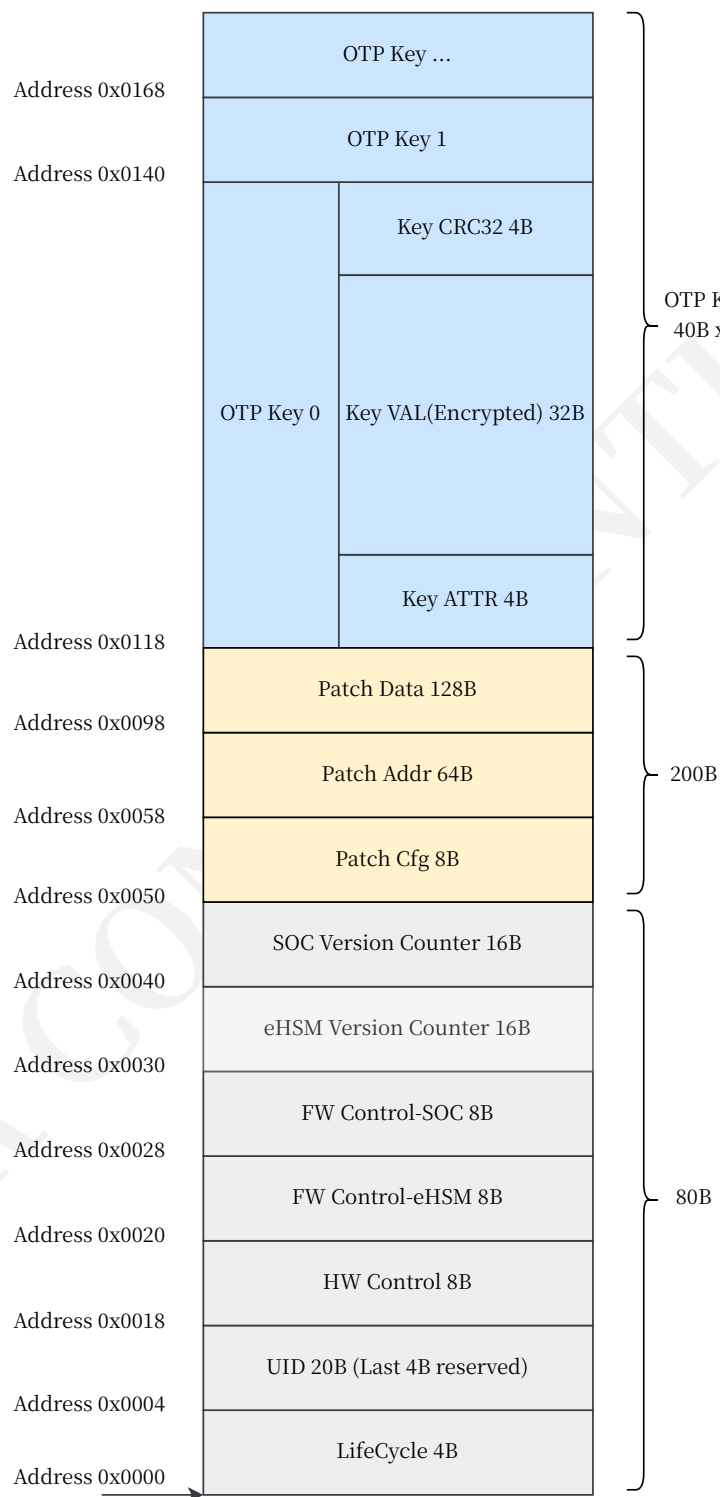


图 2 OTP 分布

其中：

- 生命周期数据(LifeCycle) 由硬件读取，如果值不正确，eHSM 的 CPU 会被 hold 住无法运行

- UID 字段共 20 字节，其中前 16 字节为有效的 UID 数据，后 4 字节为 CRC32 校验值。
- 控制数据会在上电时被硬件加载到系统寄存器中，软件直接从系统寄存器获取相应的配置值
- Patch 配置表数据在上电时由硬件 Boot 模块读取，使能后会写入到硬件 IPatch 模块
- 密钥数据会在上电时被硬件解密到 KMU 中
- 具体细节请参考 [1] 的“Secure Storage”章节。

在 TEST\_MODE 和 DEVELOP\_MODE 状态下，BootLoader 支持 OTP 数据的读写，以方便开发与测试。参考 [OTP 读命令 18](#) 和 [OTP 写命令 20](#)。OTP Patch 配置表属于 OTP 区域，读写限制请参考 [章节 3.1.1](#) 和 [表 18](#)。

### 3.1 生命周期

eHSM 支持以下生命周期<sup>3</sup>:

表 3 生命周期编码

| 生命周期             | 值          | Owner         | 说明               |
|------------------|------------|---------------|------------------|
| TEST_MODE        | 0x00000000 | Chip Vendor   | 芯片测试             |
| DEVELOP0_MODE    | 0x42818414 | Chip Vendor   | 固件开发             |
| DEVELOP_MODE     | 0x4681A416 | Device Vendor | 设备开发/测试/制造       |
| MANUFACTURE_MODE | 0x46C1A416 | Device Vendor | 设备开发/测试/制造       |
| USER_MODE        | 0x57C1EC96 | Customer      | 用户使用设备           |
| DEBUG_MODE       | 0xD7C5FCDE | Chip Vendor   | 芯片/设备调试          |
| DESTROY_MODE     | 0xFFFFFFFF | -             | 芯片/设备不可用         |
| UNDEFINE_MODE    | 其它值        | -             | 等同于 DESTROY_MODE |

#### 3.1.1 OTP Patch 配置表读写权限

OTP Patch 配置表位于 OTP 地址 0x50 至 0x117，包括 Patch Cfg、Patch Addr 和 Patch Data。该区域是否允许通过 BootLoader 的 OTP 读写命令修改，取决于当前生命周期：

表 4 OTP Patch 配置表读写权限

| 生命周期             | OTP 读命令 | OTP 写命令 | 说明                                                      |
|------------------|---------|---------|---------------------------------------------------------|
| TEST_MODE        | 允许      | 允许      | 用于芯片测试和 Patch 配置验证。写入后需要重新上电或复位，硬件才会重新加载 OTP Patch 配置表。 |
| DEVELOP_MODE     | 允许      | 允许      | 用于设备开发、测试和制造阶段配置 Patch 区域。写入受 OTP 单向写入特性限制。             |
| MANUFACTURE_MODE | 禁止      | 禁止      | BootLoader OTP 读写命令不允许访问，包括 Patch 区域。                   |
| USER_MODE        | 禁止      | 禁止      | BootLoader OTP 读写命令不允许访问，包括 Patch 区域。                   |
| DEBUG_MODE       | 禁止      | 禁止      | BootLoader OTP 读写命令不允许访问，包括 Patch 区域。                   |
| DESTROY_MODE     | 禁止      | 禁止      | 芯片/设备不可用。                                               |
| UNDEFINE_MODE    | 禁止      | 禁止      | 等同于 DESTROY_MODE。                                       |

<sup>3</sup>可参考 [1] 的“4 Security Lifecycle”章节

## 3.2 UID

UID 字段总共占用 20 字节，其中前 16 字节为 UID 数据，后 4 字节为前 16 字节的 CRC32 校验。生产时应确保每个 eHSM 写入的 16 字节 UID 是唯一的，否则一些鉴权操作会存在安全风险。

其中 CRC32 算法的多项式为：0x04C11DB7，初始值为 0xFFFFFFFF。

UID 由软件读取并使用。

## 3.3 OTP 控制字段

注：

- 本节描述的控制字段在 OTP 中都以小端序存储；
- 硬件控制字段请参考 [1]。

表 5 FW Control-eHSM 字段说明

| Bit.idx | Name               | Description                                                                                                                                                                                                                                                                          |
|---------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 63:47   | Reserved           |                                                                                                                                                                                                                                                                                      |
| 46:46   | UartLogSwitch      | UART LOG 打印功能开关 <ul style="list-style-type: none"><li>• 0b0: 开启 UART LOG 打印</li><li>• 0b1: 关闭 UART LOG 打印</li></ul>                                                                                                                                                                  |
| 45:42   | WatchdogLevel      | Watchdog 定时级别参数 <ul style="list-style-type: none"><li>• 0b0000: 关闭 Watchdog</li><li>• Other: 作为指数 N</li></ul> Watchdog 的超时时间将被配置为 $2^N \times 10 \text{ ms}$                                                                                                                         |
| 41:40   | PatchEnable        | eHSM 补丁使能 <ul style="list-style-type: none"><li>• 0b00: 无补丁</li><li>• 0b01: 有补丁</li><li>• 0b10: 有补丁</li><li>• 0b11: 无补丁</li></ul>                                                                                                                                                    |
| 39:36   | EhsmCodeUpgradeAlg | eHSM 安全升级镜像验签算法选择: <ul style="list-style-type: none"><li>• 0b0000: SHA256-RSA2048-PSS</li><li>• 0b0001: SM3-SM2</li><li>• 0b0100: AES128-CMAC</li><li>• 0b0101: SM4-CMAC</li><li>• 0b0110: SHA256-RSA3072-PSS</li><li>• 0b0111: SHA256-ECDSA-SECP256R1</li><li>• Other: 保留</li></ul> |
| 35:32   | EhsmCodeVerifyAlg  | eHSM 安全启动镜像验签算法选择: <ul style="list-style-type: none"><li>• 0b0000: SHA256-RSA2048-PSS</li><li>• 0b0001: SM3-SM2</li><li>• 0b0010: AES128-CMAC</li><li>• 0b0011: SM4-CMAC</li></ul>                                                                                                   |

| Bit.idx | Name              | Description                                                                                                                                                                                                                                                                |
|---------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |                   | <ul style="list-style-type: none"> <li>• 0b0110: SHA256-RSA3072-PSS</li> <li>• 0b0111: SHA256-ECDSA-SECP256R1</li> <li>• Other: 保留</li> </ul>                                                                                                                              |
| 31:15   | Reserved          |                                                                                                                                                                                                                                                                            |
| 14:13   | RandomClockLevel  | RANDOM CLOCK 配置级别 <sup>4</sup> : <ul style="list-style-type: none"> <li>• 0b00: 关闭</li> <li>• 0b01: randomly gate 1 clock cycle in 32 cycles</li> <li>• 0b10: randomly gate 1 clock cycle in 16 cycles</li> <li>• 0b11: randomly gate 1 clock cycle in 8 cycles</li> </ul> |
| 12:11   | BootOsccaSwitch   | 国密认证特性开关: <ul style="list-style-type: none"> <li>• 0b00: 关闭</li> <li>• 0b10: 开启</li> <li>• 0b01: 开启</li> <li>• 0b11: 关闭</li> </ul> 注: 应该仅在进行国密认证时开启此功能。                                                                                                                    |
| 10:8    | Reserved          |                                                                                                                                                                                                                                                                            |
| 7:6     | BootSelfTest      | Boot 自检级别 <ul style="list-style-type: none"> <li>• 0b00: 关闭自检</li> <li>• 0b10 或 0b01: 基本自检</li> <li>• 0b11: 完整自检</li> </ul>                                                                                                                                                |
| 5       | SensorAlarmAction | 当 Sensor 告警时, BL 进行以下处理: <ul style="list-style-type: none"> <li>• 0b0: Hold 住 CPU 直到告警结束</li> <li>• 0b1: 清除 OTP 中的所有密钥, 然后 Hold 住 CPU 直到告警结束</li> </ul> 此配置对应的操作仅在 BootOsccaSwitch 开启时有效, BootOsccaSwitch 关闭时 BL 忽略 Sensor 告警。                                             |
| 4:0     | Reserved          |                                                                                                                                                                                                                                                                            |

表 6 FW Control-SOC 字段说明

| Bit.idx | Name          | Description                                                                                                                                                                                                                |
|---------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 63:8    | Reserved      |                                                                                                                                                                                                                            |
| 7:4     | SocUpgradeAlg | SOC 安全升级镜像算法选择: <ul style="list-style-type: none"> <li>• 0b0000: SHA256-RSA2048-PSS</li> <li>• 0b0001: SM3-SM2</li> <li>• 0b0100: AES128-CMAC</li> <li>• 0b0101: SM4-CMAC</li> <li>• 0b0110: SHA256-RSA3072-PSS</li> </ul> |

<sup>4</sup>此处配置由 BL 读取并设置。注意, 在 OTP 的 HW 控制字段也有一个 random clock 使能配置, 它与此配置的关系如下:

- HW 控制字段使能, 则此配置无效, BL 虽然会读取并配置, 但不会起作用; HW 的使能固定配置为 1/32
- HW 控制字段关闭, 则此配置生效, 由 BL 读取并配置



| Bit.idx | Name       | Description                                                                                                                                                                                                                                                                   |
|---------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |            | <ul style="list-style-type: none"> <li>0b0111: SHA256-ECDSA-SECP256R1</li> <li>Other: 保留</li> </ul>                                                                                                                                                                           |
| 3:0     | SocBootAlg | SOC 安全启动镜像密钥算法选择: <ul style="list-style-type: none"> <li>0b0000: SHA256-RSA2048-PSS</li> <li>0b0001: SM3-SM2</li> <li>0b0010: AES128-CMAC</li> <li>0b0011: SM4-CMAC</li> <li>0b0110: SHA256-RSA3072-PSS</li> <li>0b0111: SHA256-ECDSA-SECP256R1</li> <li>Other: 保留</li> </ul> |

### 3.4 版本计数器

OTP 中包含了 eHSM FW 和 SOC FW 的版本计数器 (Version Counter)。Version Counter 是一个 16 字节的数据，用于管理固件的版本升级，基于 OTP 单向修改的特性，对这个数有一定的要求。

版本计数器在 OTP 中的偏移如下：eHSM FW Version Counter 位于 0x030，SOC FW Version Counter 位于 0x040。

假设 OTP 的默认值为 0，只能修改为 1，则：

- 16 字节的数据取反后作为一个小端序的大数
- 大数从 0 开始向上增加，每次从最低的为 0 的位变为 1
- 升级时 Version Counter 必须大于等于 OTP 中记录的 Version Counter 才能成功
- 总共支持 128 个版本

例：

表 7 Version Counter 编码示例

| 版本  | 大数值                                   | OTP 存储值                          |
|-----|---------------------------------------|----------------------------------|
| 1   | 0x00000000000000000000000000000001    | 01000000000000000000000000000000 |
| 2   | 0x00000000000000000000000000000003    | 03000000000000000000000000000000 |
| 15  | 0x00000000000000000000000000000007FFF | FF7F0000000000000000000000000000 |
| 64  | 0x0000000000000000FFFFFFFFFFFFFFFF    | FFFFFFFFFFFFFFFF0000000000000000 |
| 128 | 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF    | FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF |

### 3.5 Patch 机制

#### 3.5.1 硬件支持

硬件的 IPatch 模块支持以下特性：

- 将上电过程中硬件 Boot 模块从 OTP 读取的 Patch 数据保存在 IPatch 模块内

- 将 CPU 对 IBUS 上指定地址读取的 32 位指令替换为另一个 32 位指令或两个 16 位指令，替换范围请参考 [1] 的 “Ipatch” 章节
- 支持 32 个 Patch 入口，Patch 的地址必须 32 位对齐

注：

- 硬件控制字段包含一个 2 比特的 IPatch 使能总开关，只有该总开关配置为使能硬件 IPatch 功能时，OTP Patch 配置表才会生效。请参考 [1]。
- IPatch 模块仅支持对 32 位对齐的地址打 Patch，因此，如果替换 32 位指令但地址不是 32 位对齐时，需要消耗 2 个 Patch 点来替换。
- 由于 IPatch 本质上是对 IROM 数据进行替换，因此理论上可以将任何有效范围内的 IROM 地址的指令替换为其它指令。

### 3.5.2 OTP Patch 配置表

OTP 包含 Patch 配置表，共支持 32 个 Patch 行，可以实现最多 32 个 words 的指令数据替换，替换的地址必须 word 对齐。对于  $0 \leq i \leq 31$ ，每个 Patch 行由一个 2 比特 Patch Cfg[2i+1:2i]，一个 16 比特 Patch Addr[16i+15:16i]，以及一个 32 比特 Patch Data[32i+31:32i] 组成。

以下是 OTP Patch 配置表：

表 8 OTP Patch 配置表

| 参数         | OTP 偏移(字节) | 说明                                                                                                                                                                                                                                                                                                                  |
|------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Patch Cfg  | 0x50       | 共 8 字节，包含 32 个 Patch 行使能，每 2 比特对应一个 Patch 行的使能开关，对于 $0 \leq i \leq 31$ ： <ul style="list-style-type: none"> <li>• 当 Patch Cfg[2i+1:2i]<sup>5</sup> = 2b00 或 2b11 时，该 Patch 行无效</li> <li>• 当 Patch Cfg[2i+1:2i] = 2b01 或 2b10 时，该 Patch 行生效</li> </ul>                                                               |
| Patch Addr | 0x58       | 共 64 字节，包含 32 个地址偏移，每 16 比特对应一个被替换的 IROM 地址偏移，对于 $0 \leq i \leq 31$ ，实际对应被替换的 IROM 地址计算规则为： <p>实际被替换的 IROM 地址 = IROM 起始地址 + (Patch Addr[16i+15:16i] &lt;&lt; 2)</p> <p>例如若 IROM 起始地址为 0x10000000，第 0 个 Patch Addr[15:0] = 0x0010 时，则第 0 个 Patch 行实际被替换的 IROM 地址为 0x10000000 + (0x0010 &lt;&lt; 2) = 0x10000040。</p> |
| Patch Data | 0x98       | 共 128 字节，包含 32 个 words 待替换的指令数据，对于 $0 \leq i \leq 31$ ，Patch Data[32i+31:32i] 代表将要写入 Patch Addr[16i+15:16i] 指向的 IROM 地址的数据。                                                                                                                                                                                         |

注：当硬件控制字段配置 IPatch 模块使能（请参考 [1]），且 OTP Patch Cfg 配置开启，同时对应的 OTP Patch Addr 和 Patch Data 有效时，才会进行指令数据的替换。

下图是 CPU 读取 IROM 指令的流程：

<sup>5</sup>本表中 i 指 Patch 行序号，例如当 i = 0 表示 Patch Cfg[1:0] 比特；当 i = 31 表示 Patch Cfg[63:62] 比特。其他参数中的 i 同理。

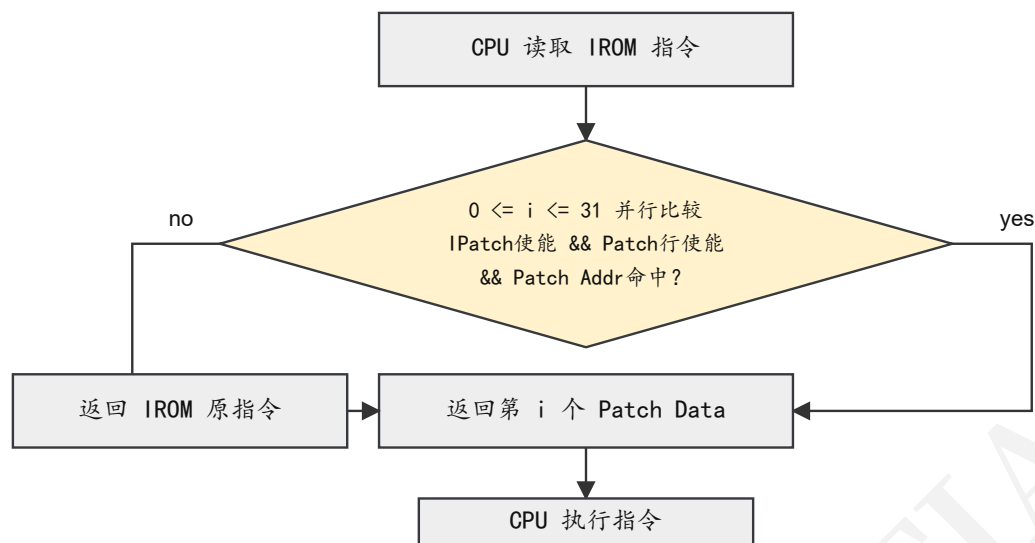


图 3 CPU 读取指令流程

### 3.6 OTP 密钥排布

OTP 密钥存储在 OTP 偏移 0x118 开始的位置，以下为 OTP 的密钥排布（支持定制位置及复用）：

表 9 OTP 密钥排布表

| ID | Level | OTP 偏移(字节) | 密钥用途                     |
|----|-------|------------|--------------------------|
| 0  | -     | 280        | Chip Root Key            |
| 1  | -     | 320        | Device Root Key          |
| 2  | -     | 360        | User Root Key            |
| 3  | 1     | 400        | eHSM Debug Verify Key    |
| 4  | 1     | 440        | eHSM FW Verify Key       |
| 5  | 1     | 480        | eHSM Encrypt Key         |
| 6  | 1     | 520        | eHSM Upgrade Encrypt Key |
| 7  | 1     | 560        | eHSM Upgrade Verify Key  |
| 8  | 1     | 600        | eHSM Private Key         |
| 9  | 2     | 640        | Soc Debug Verify Key     |
| 10 | 2     | 680        | Soc FW Verify Key        |
| 11 | 2     | 720        | Soc Encrypt Key          |
| 12 | 2     | 760        | Soc Upgrade Encrypt Key  |
| 13 | 2     | 800        | Soc Upgrade Verify Key   |
| 14 | 2     | 840        | Soc Private Key          |
| 15 | 2     | 880        | Secret Key               |
| 16 | 2     | 920        | User Auth Key            |

注：

- Level 1 的密钥使用 Chip Root Key 加密
- Level 2 的密钥使用 Device Root Key 加密

- Key1 (Device Root Key) 和 Key2 使用 Chip Root Key 加密，忽略 Level 配置
- Key0 (Chip Root Key) 使用 RTL KEY 加密，忽略 Level 配置
- 表格中的密钥排列为示例，支持调整顺序，甚至多个功能密钥映射到同一存储位置

相应密钥说明可参考 [1] 的 “3.4.1 eHSM Keys” 章节。

## 3.7 OTP 驱动要求

### 3.7.1 OTP 数据读取

eHSM BootLoader 在运行过程中需要读取 OTP 数据。通常情况下，eHSM 硬件设计已确保可通过总线直接读取 OTP 数据，因此无需特定驱动程序。若存在特殊情况，客户应负责实现读取 OTP 数据的驱动程序，并与 BootLoader 代码集成。

### 3.7.2 OTP 数据写入

如果 eHSM BootLoader

- 支持 `CMD-BL-OTP-WRITE` 命令
- 需要通过国密二级认证

则客户应负责实现 OTP 写入驱动，并与 BootLoader 代码集成，以满足相应功能或认证要求。

### 3.7.3 驱动 API

请参考 [2]。

## 4 密钥烧录

BootLoader 支持以下两个命令：

- [生成加密密钥 30](#)
- [对输入密钥加密 32](#)

上述命令用于生成被指定级别的根密钥加密的密钥，生成的数据为密钥密文+CRC32，用户需要自行拼接 4 字节密钥属性，然后可以使用写 OTP 命令将数据写入 OTP 中。

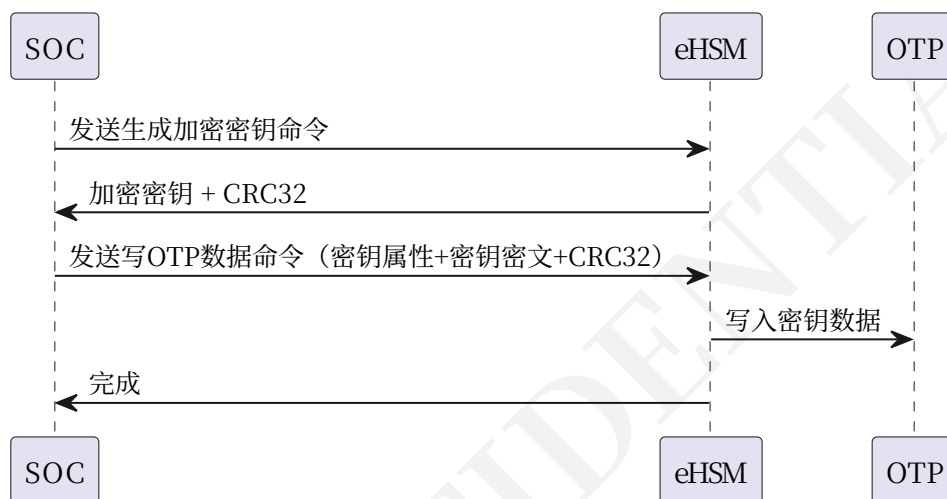


图 4 生成密钥并烧录流程

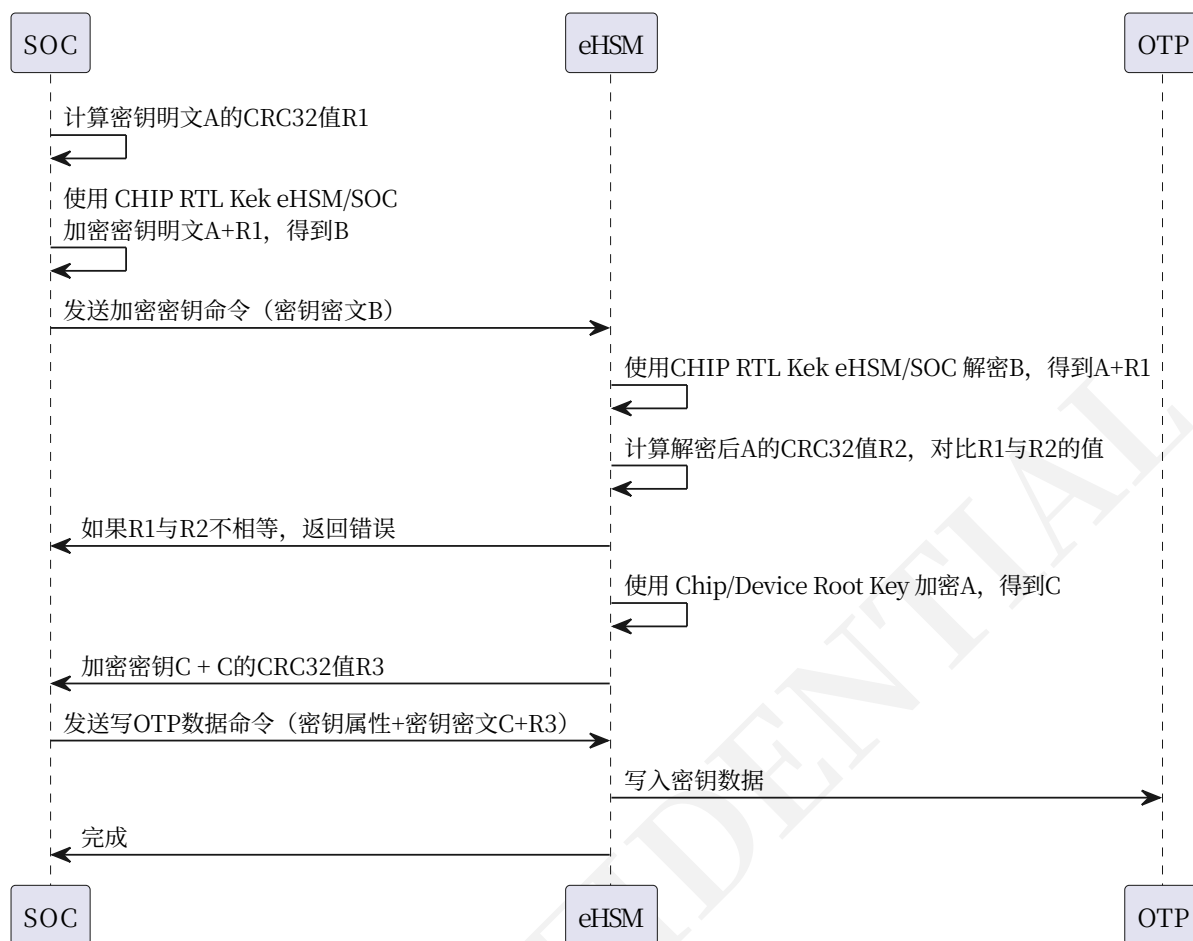


图 5 加密密钥并烧录流程

对于[对输入密钥加密 32](#) 命令，输入的密钥值需要根据 “key\_level” 被 “Chip RTL Kek eHSM” 或 “Chip RTL Kek Soc” 使用 CBC 算法加密 (IV=全 0)，使用的对称算法由 [\[1\]](#) “OTP Hardware Control Fields” 表中的 “KeyAlgSel” 字段定义。

关于 CHIP RTL Kek eHSM/SOC 密钥请参考 [\[1\]](#) 中的 “Key Management” 章节。

## 5 鉴权

一些功能需要鉴权后才可以使用：

1. 开启 eHSM 调试功能
2. 开启 SOC 调试功能
3. 固件控制功能，主要用于国密认证

### 5.1 调试鉴权

各功能对应的密钥如下表所示<sup>6</sup>

表 10 鉴权对应密钥

| 鉴权类型      | OTP 密钥         |
|-----------|----------------|
| eHSM 调试鉴权 | eHSM Debug Key |
| SOC 调试鉴权  | SOC Debug Key  |

鉴权需要用到以下命令：

- [获取挑战值命令 34](#)
- [调试鉴权命令 36](#)

基本的调试鉴权流程如下图所示：

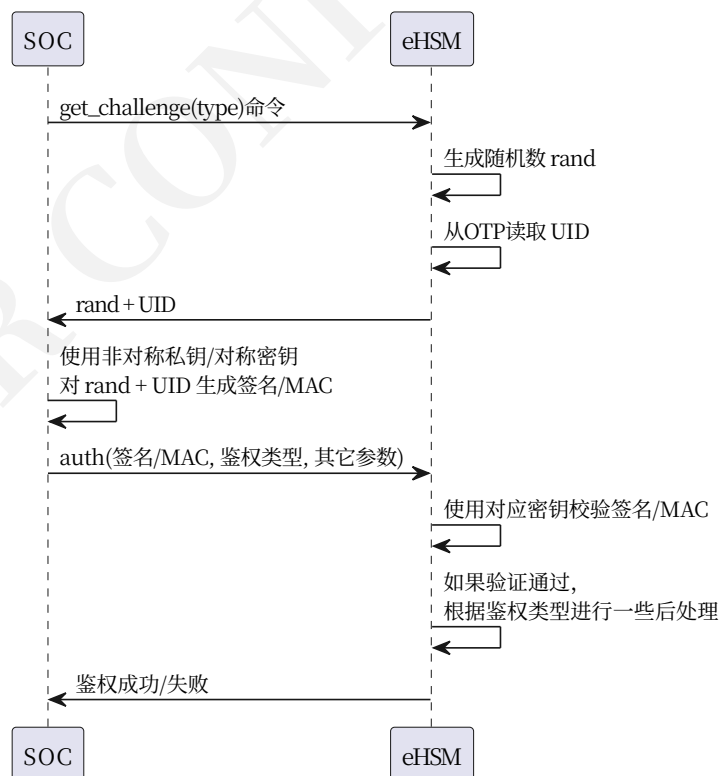


图 6 调试鉴权流程

<sup>6</sup>OTP 密钥请参考 [章节 3.6](#)

调试鉴权用于开启 eHSM 或 SOC 的调试能力，鉴权通过后，打开对应的调试端口：

- 对于 eHSM 调试，仅有一个端口
- 对于 SOC 调试，需要在调试鉴权命令中指定共计 129 个端口的使能位参数，端口对应的 bit 为 1 时打开调试，bit 为 0 的保持原状态

如果需要关闭调试端口，请使用 [关闭调试命令 38](#)，如果是 SOC 的调试端口，需在命令中指定共计 129 个端口的使能位参数，对应 bit 为 1 的端口被关闭，bit 为 0 的端口保持原状态。

## 5.2 固件控制鉴权

此鉴权命令用于国密认证相关的功能鉴权，用于满足一些认证要求，包含以下功能：

- 鉴权后清除用户密钥
- 鉴权后更新 HSM 固件版本号
- 鉴权后自毁

请参考 [\[3\]](#) 获取详细的使用方式。



## 6 安全启动

本章节描述了 eHSM 安全启动设计, 包括 eHSM Bootloader 提供的安全启动能力, 和对 SOC 侧的流程建议。

### 6.1 安全启动交互流程

#### 6.1.1 并行启动模式

并行启动模式下, eHSM 与 SOC 分别启动自己的 BootLoader, 然后 SOC Bootloader 发送命令验证 SOC FW Image 的有效性 (包含解密), 以及最后验证 eHSM FW 的有效性, 并各自启动自己的 FW。

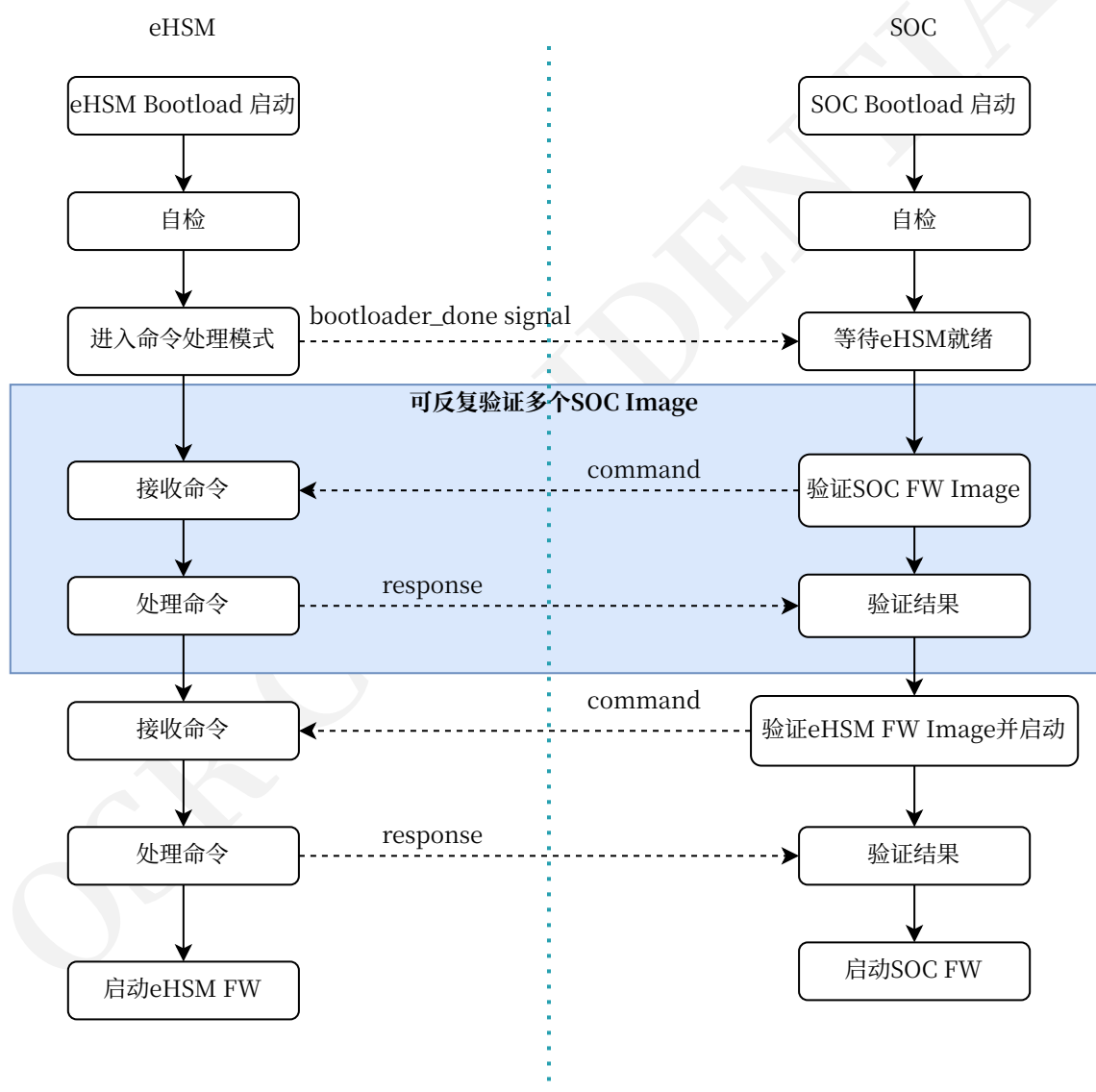


图7 并行启动流程

## 6.2 部署说明

### 6.2.1 eHSM FW 部署

eHSM FW 可以支持两种类型的代码部署方式：

1. 基于 IRAM 的部署方式：FW 镜像被解封到 eHSM 内部的 IRAM 中执行
2. 基于 NVM 的部署方式：FW 镜像存储于外部 NVM 中，并被 eHSM 的 CPU 校验后直接读取指令并执行

基于 IRAM 的部署：

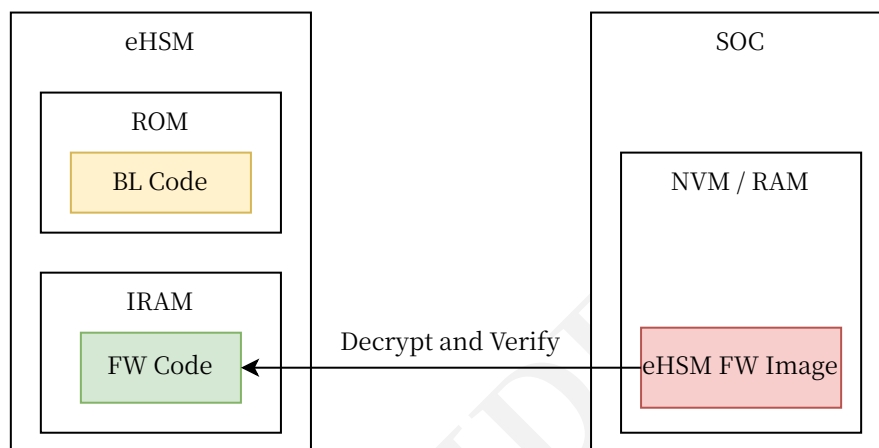


图 8 eHSM FW 基于 IRAM 的部署方式

在 IRAM 部署方式下，由 SoC 将 Image 数据准备在存储器中，再通知 eHSM 进行验证。eHSM 将镜像数据一边读取一边解密，并将解密数据输出到内部 IRAM 中，然后校验数据的签名，最后回复 SoC 校验的结果。如果 SoC 在命令里指定跳转到 FW，则 eHSM 跳转到 IRAM 中的 FW 入口地址执行。

基于 NVM 的部署：

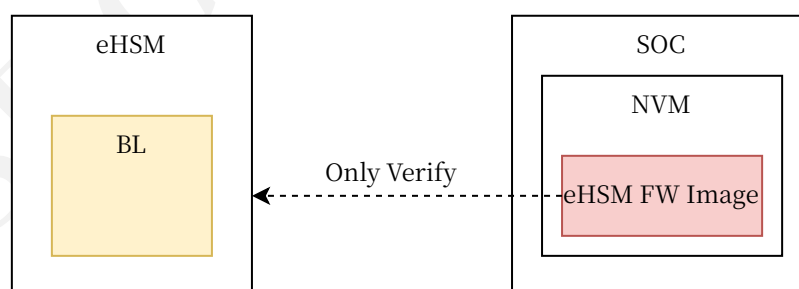


图 9 eHSM FW 基于 NVM 的部署方式

在 NVM 部署方式下，由 SOC 将 Image 数据准备在 NVM 存储器中，再通知 eHSM 进行验证。eHSM 读取镜像数据并直接校验数据的签名，最后回复 SoC 校验的结果。如果 SoC 在命令里指定跳转到 FW，则 eHSM 跳转到 NVM 中的 FW 入口地址执行。

这种部署模式需要满足以下条件：

- eHSM CPU 可从 NVM 中读取镜像数据（直接通过数据总线读取或通过驱动程序读取）

- eHSM CPU 可直接通过指令总线从 NVM 中读取指令数据
- 镜像只包含签名值，不能加密

## 6.2.2 SOC FW 部署

SOC FW 也可以支持两种类型的代码部署方式：

1. 基于 RAM 的部署方式：FW 镜像被解封 SoC 的 RAM 中执行
2. 基于 NVM 的部署方式：FW 镜像存储 NVM 中，并被 eHSM 校验后直接由 SoC 执行

基于 RAM 的部署：

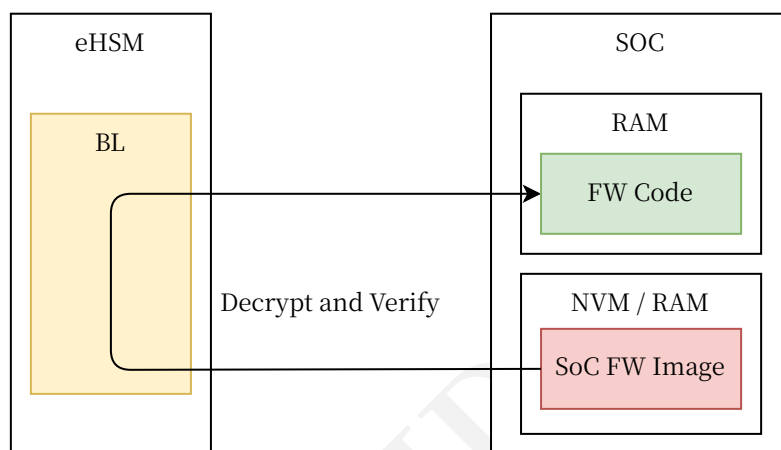


图 10 SoC FW 基于 RAM 的部署方式

在 RAM 部署方式下，由 SoC 将 Image 数据准备在存储器中，再通知 eHSM 进行验证。eHSM 将镜像数据读取并解密到 SoC 的 RAM 中，并校验数据的签名，最后回复 SoC 校验的结果。

在这种部署模式下，eHSM 支持将 SoC FW 镜像解密到原位置并校验，这要求镜像必须存储在 RAM 中。

基于 NVM 的部署：

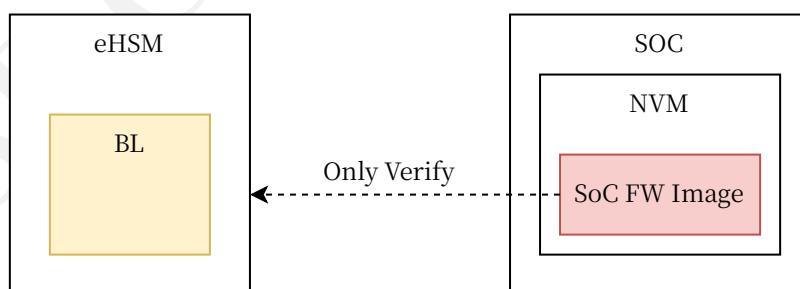


图 11 SoC FW 基于 NVM 的部署方式

在 NVM 部署方式下，由 SOC 将 Image 数据准备在 NVM 存储器中，再通知 eHSM 进行验证。eHSM 将镜像数据读取并直接校验数据的签名，最后回复 SoC 校验的结果。

这种部署模式需要满足以下条件：

- eHSM CPU 可从 SOC 传入的地址从 NVM 中读取镜像数据
- 镜像只包含签名值，不能加密

### 6.3 镜像格式

eHSM 和 SOC 的安全启动镜像遵从以下格式：

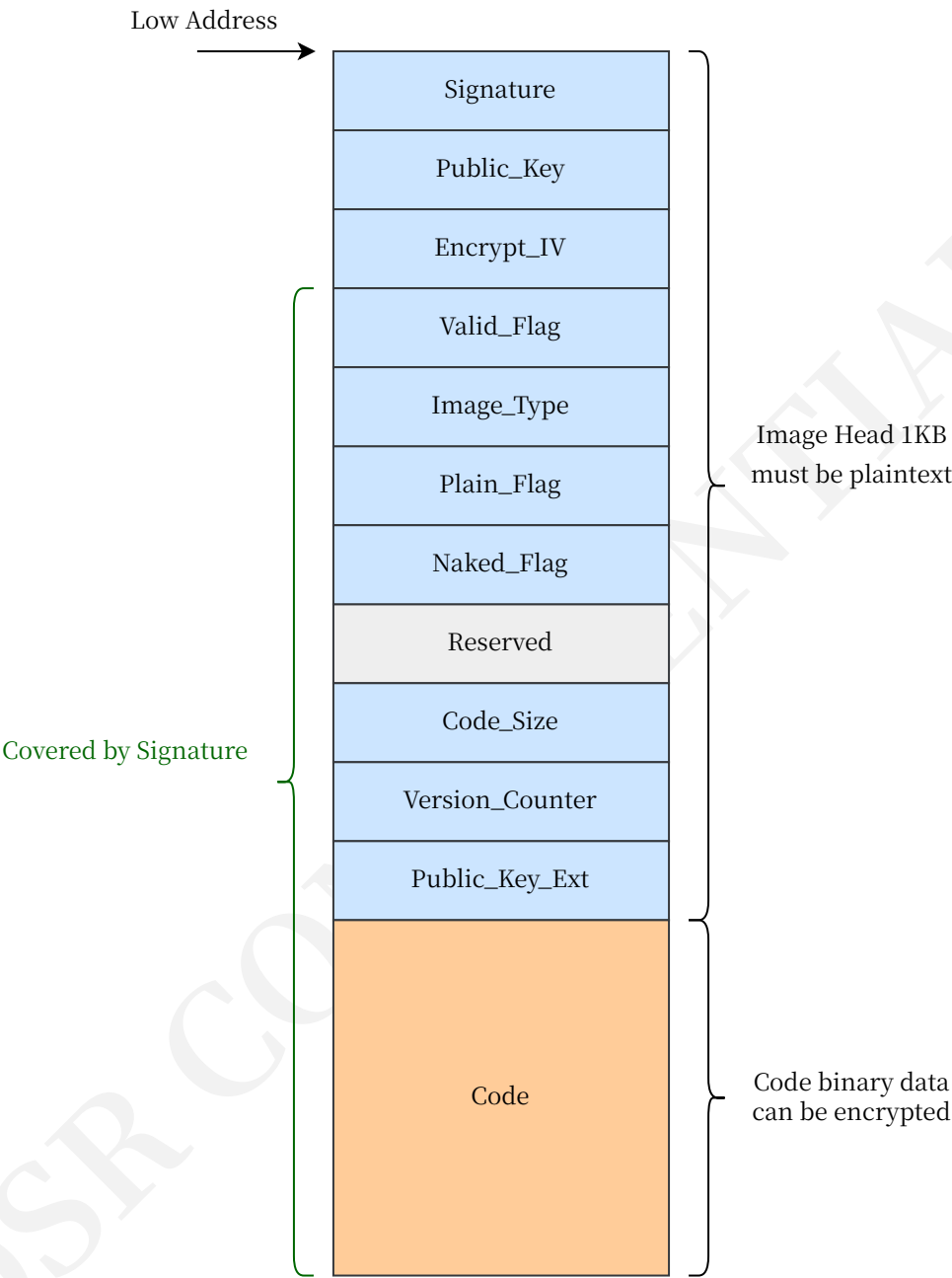


图 12 镜像格式示意图

表 11 安全启动镜像字段说明

| 字段        | 偏移(字节) | 大小(字节) | 说明                                                                                                                                                                                                     |
|-----------|--------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Signature | 0      | 256    | 有效的数据长度由 OTP 中的 “EhsmCodeVerifyAlg” 或 “SocBootAlg” 定义的算法决定： <ul style="list-style-type: none"><li>• AES128/SM4-CMAC: 长度为 16 字节</li><li>• RSA2048: 长度为 256 字节</li><li>• RSA3072: 签名值的前 256 字节</li></ul> |

| 字段              | 偏移(字节) | 大小(字节)    | 说明                                                                                                                                                                                                                                                                                        |
|-----------------|--------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 |        |           | <ul style="list-style-type: none"> <li>SM2: 长度为 64 字节</li> <li>ECC-SECP256R1: 长度为 64 字节</li> </ul>                                                                                                                                                                                        |
| Public_Key      | 256    | 320       | Public_Key 用于非对称算法。有效的长度由具体算法决定: <ul style="list-style-type: none"> <li>AES128/SM4 CMAC: 此字段无效</li> <li>RSA2048: 64 字节的 RSA(e) + 256 字节的 RSA(n)</li> <li>RSA3072: 签名值的后 128 字节 + RSA(e) 64 字节</li> <li>SM2: 长度为 65 字节, 第一个字节 “0x04” 表示公钥值未压缩</li> <li>ECC-SECP256R1: 长度为 64 字节</li> </ul> |
| Encrypt_IV      | 576    | 16        | AES/SM4 CMAC 算法的 IV 值                                                                                                                                                                                                                                                                     |
| Valid_Flag      | 592    | 4         | 代码是否有效标志: <ul style="list-style-type: none"> <li>0x8E97645D: 代码有效</li> <li>其它值: 代码无效</li> </ul>                                                                                                                                                                                           |
| Image_Type      | 596    | 1         | 镜像类型: <ul style="list-style-type: none"> <li>0: eHSM 安全启动镜像(必须使用 eHSM 密钥加密/签名)</li> <li>1: 使用 SOC 密钥加密/签名的 SOC 安全启动镜像</li> <li>2: 使用 eHSM 密钥加密/签名的 SOC 安全启动镜像</li> <li>3: 使用 eHSM 密钥加密/签名的 eHSM Patch 镜像<sup>7</sup></li> </ul>                                                           |
| Plain_Flag      | 597    | 1         | 代码部分是否明文: <ul style="list-style-type: none"> <li>0: 代码部分是密文</li> <li>1: 代码部分是明文</li> </ul>                                                                                                                                                                                                |
| Naked_Flag      | 598    | 1         | 是否需要验证签名: <ul style="list-style-type: none"> <li>0: 非裸镜像数据</li> <li>1: 裸镜像数据, 除了此字段、Image_Type、Valid_Flag 字段和 Code 字段, 其它字段值都为全 0</li> </ul> 注意: 仅在 TEST_MODE 和 DEVELOP_MODE 生命周期状态下支持裸镜像, 主要用于简化开发和验证; 其它状态下会报错。                                                                         |
| Reserved        | 599    | 5         | 保留 (应当设置为全 0)                                                                                                                                                                                                                                                                             |
| Code_Size       | 604    | 4         | 代码区域的大小                                                                                                                                                                                                                                                                                   |
| Version_Counter | 608    | 16        | 128 位的单向版本计数                                                                                                                                                                                                                                                                              |
| Public_Key_Ext  | 624    | 400       | 当验签密钥为 RSA3072 时, 此字段包含 384 字节的 RSA(n)值, 后跟全 0; 其它验签密钥时此字段全部为 0                                                                                                                                                                                                                           |
| Code            | 1024   | Code_Size | 代码区域, 明文或密文                                                                                                                                                                                                                                                                               |

## 6.4 验证命令定义

请参考 [镜像验证命令 28](#)。

## 6.5 验证流程

eHSM 对镜像的验证流程如下图:

<sup>7</sup>仅 bootloader 支持加载此类镜像

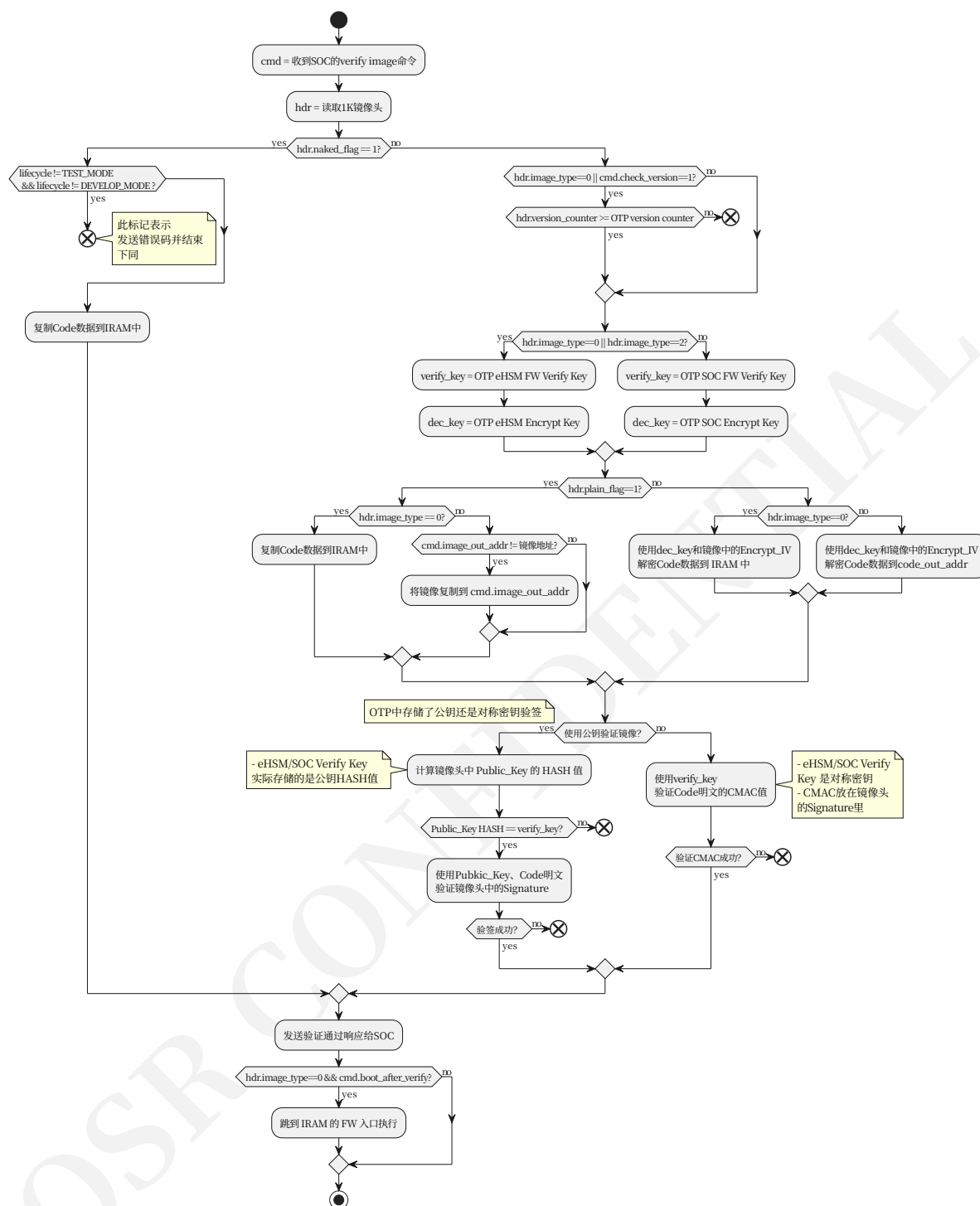


图 13 镜像的验证流程

## 6.6 配置说明

### 6.6.1 算法配置

支持以下算法：

- 验证算法：参考表 表 5 中的“EhsmCodeVerifyAlg”字段以及表 表 6 中的“SocBootAlg”字段

- 加密算法：AES128-CBC/SM4-CBC（根据验证算法使用国际还是国密，决定使用 AES128 还是 SM4）

验证算法使用的密钥在 OTP 中存储，eHSM FW 使用的密钥为“eHSM FW Verify Key”和“eHSM Encrypt Key”，SOC FW 使用的密钥为“SOC FW Verify Key”和“SOC Encrypt Key”，请参考表 表 9。

### 6.6.2 存储访问方式配置

支持两种外部存储访问方式：

- SOC RAM 访问，由硬件 memory remap 机制提供支持；
- SOC NVM 访问，需要客户提供读取 SOC NVM 数据的驱动并集成到 eHSM BootLoader 中。这种方式无法支持 DMA，因此性能较差。

以上两种访问方式由配置指定二选一，生成 BootLoader 时就已确定。

## 7 安全固件升级

本章节描述了 eHSM 安全固件升级功能的设计。

### 7.1 升级镜像制作

升级镜像是相对于 eHSM/SOC 的安全启动镜像而言，即在 eHSM/SOC 的安全启动镜像上附加了一层加密和签名。

升级镜像的制作分三步：

- 1. 生成明文安全启动镜像，使用 eHSM/SOC Verify Key 进行签名
- 2. 将明文安全启动镜像整个使用 eHSM/SOC Upgrade Encrypt Key 进行加密
- 3. 增加升级镜像头，并使用 eHSM/SOC Upgrade Verify Key 进行签名

参考下图：

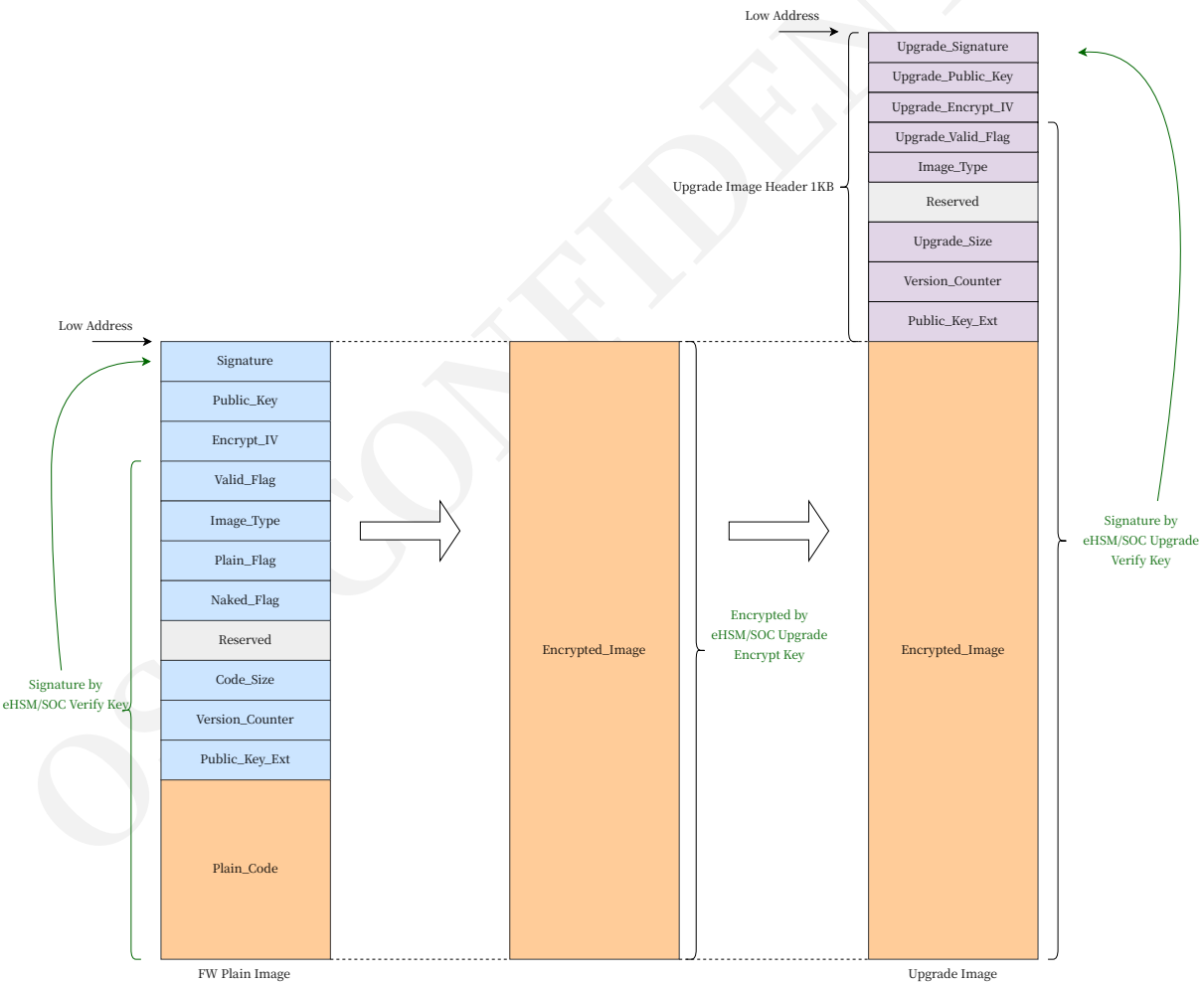


图 14 制作升级镜像

表 12 升级镜像字段说明



| 字段                 | 偏移(字节) | 大小(字节)       | 说明                                                                                                                                                                                                                                                                                                  |
|--------------------|--------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Upgrade_Signature  | 0      | 256          | 有效的数据长度由 OTP 中的 “EhsmCodeUpgradeAlg” 或 “SocUpgradeAlg” 定义的算法决定： <ul style="list-style-type: none"> <li>• AES128/SM4 CMAC: 长度为 16 字节</li> <li>• RSA2048: 长度为 256 字节</li> <li>• RSA3072: 签名值的前 256 字节</li> <li>• SM2: 长度为 64 字节</li> <li>• ECC-SECP256R1: 长度为 64 字节</li> </ul>                        |
| Upgrade_Public_Key | 256    | 320          | Public_Key 用于非对称算法。有效的长度由具体算法决定： <ul style="list-style-type: none"> <li>• AES128/SM4 CMAC: 此字段无效</li> <li>• RSA2048: 64 字节的 RSA(e) + 256 字节的 RSA(n)</li> <li>• RSA3072: 签名值的后 128 字节 + RSA(e) 64 字节</li> <li>• SM2: 长度为 65 字节, 第一个字节 “0x04” 表示公钥值未压缩</li> <li>• ECC-SECP256R1: 长度为 64 字节</li> </ul> |
| Upgrade_Encrypt_IV | 576    | 16           | AES/SM4 CMAC 算法的 IV 值                                                                                                                                                                                                                                                                               |
| Upgrade_Valid_Flag | 592    | 4            | 代码是否有效标志： <ul style="list-style-type: none"> <li>• 0x71689BA2: 镜像有效</li> <li>• 其它值: 镜像无效</li> </ul>                                                                                                                                                                                                 |
| Image_Type         | 596    | 1            | 镜像类型： <ul style="list-style-type: none"> <li>• 0: eHSM 升级镜像（必须使用 eHSM Upgrade 密钥加密/签名）</li> <li>• 1: SOC 升级镜像，使用 SOC Upgrade 密钥加密/签名</li> <li>• 2: SOC 升级镜像，使用 eHSM Upgrade 密钥加密/签名</li> <li>• 3: eHSM Patch 升级镜像，使用 eHSM Upgrade 密钥加密/签名</li> </ul>                                                |
| Reserved           | 597    | 7            | 保留 (应当设置为全 0)                                                                                                                                                                                                                                                                                       |
| Upgrade_Size       | 604    | 4            | 加密镜像区域的大小                                                                                                                                                                                                                                                                                           |
| Version_Counter    | 608    | 16           | 128 位的单向版本计数                                                                                                                                                                                                                                                                                        |
| Public_Key_Ext     | 624    | 400          | 当验签密钥为 RSA3072 时，此字段包含 384 字节的 RSA(n)值，后跟全 0；其它验签密钥时此字段全部为 0                                                                                                                                                                                                                                        |
| Encrypted_Image    | 1024   | Upgrade_Size | 加密的代码镜像                                                                                                                                                                                                                                                                                             |

## 7.2 升级镜像安装

安装升级镜像的命令请参考 [固件升级命令 26](#)。

eHSM 升级流程与制作流程相反：

1. 根据升级镜像中的 “Image\_Type” 字段，使用 “eHSM Upgrade Verify Key” 或 “SOC Upgrade Verify Key” 验证升级镜像
2. 根据升级镜像中的 “Image\_Type” 字段，使用 “eHSM Upgrade Encrypt Key” 或 “SOC Upgrade Encrypt Key” 解密镜像的 “Encrypt\_Image” 字段，得到固件明文镜像

- 最后, 根据固件明文镜像中的“Plain\_Flag”字段, 决定是否要加密固件镜像的“Plain\_Code”字段; 如果需要加密, 根据固件明文镜像中的“Image\_Type”字段, 决定使用“eHSM Encrypt Key”还是“SOC Encrypt Key”进行加密

升级完成后的固件镜像是安全启动镜像, SOC 端应当使用 eHSM 的镜像验证命令对安全启动镜像进行验证, 成功后由 SOC 端负责存储, 并在后续启动时使用中提供给 eHSM 进行验证。

### 7.2.1 算法配置

支持以下算法:

- 验证算法: 参考表 表 5 中的“EhsmCodeUpgradeAlg”字段以及表 表 6 中的“SocUpgradeAlg”字段
- 加密算法: AES128-CBC/SM4-CBC (根据验证算法使用国际还是国密, 决定使用 AES128 还是 SM4)

验证算法使用的密钥在 OTP 中存储, eHSM FW 使用的密钥为“eHSM FW Upgrade Verify Key”和“eHSM Upgrade Encrypt Key”, SOC FW 使用的密钥为“SOC FW Upgrade Verify Key”和“SOC Upgrade Encrypt Key”, 请参考表 表 9。

## 8 补丁机制

注：本章描述的是由 BootLoader 验证并加载到 IRAM 的软件 Patch 镜像方案，不适用于 [章节 3.5](#) 中描述的硬件 OTP ROM Patch 机制。硬件 OTP ROM Patch 由硬件上电读取 OTP Patch 配置表并在取指令命中时替换指令，不需要交付或加载本章所述的软件 Patch 镜像。若项目未交付软件 Patch 镜像，则本章方案不支持使用。

### 8.1 硬件支持

硬件的 Ipatch 模块支持以下特性：

- 将 CPU 对 IBUS 上指定地址读取的 32 位指令替换为另一个指令
- 支持 16 个 patch 入口，patch 的地址必须 32 位对齐

具体细节请参考 [\[1\]](#) 的“Ipatch”章节。

### 8.2 软件方案

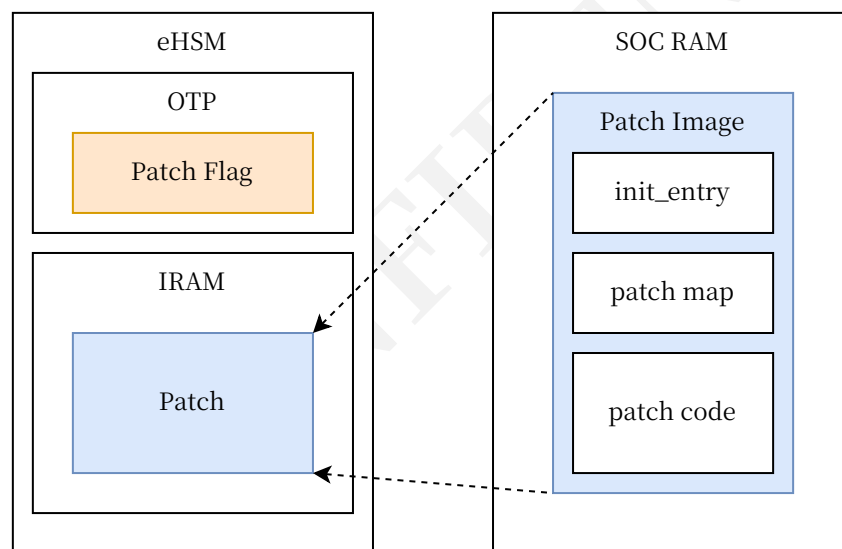


图 15 软件补丁方案概览

主要流程如下：

1. 使用[安全启动 6](#) 相同的镜像格式以保证安全
2. 在 OTP 中配置 patch 标记，Bootloader 启动后识别标记并自动验证 SOC RAM 中的 patch 镜像，然后加载到 IRAM 中
3. Bootloader 随后调用 patch 中的 `init_entry`，向 Ipatch 模块注册补丁，并注册 CPU 异常处理函数
4. 当代码执行到匹配的地址时，指令被 Ipatch 模块替换，CPU 执行被替换指令（通常被替换成 ECALL，触发异常处理函数）
5. 异常处理函数中设置 MEPC（异步处理返回后会执行的地址）为补丁函数的地址，退出异常处理函数后 CPU 会执行补丁函数

## 8.2.1 补丁加载

下图是补丁的加载流程：

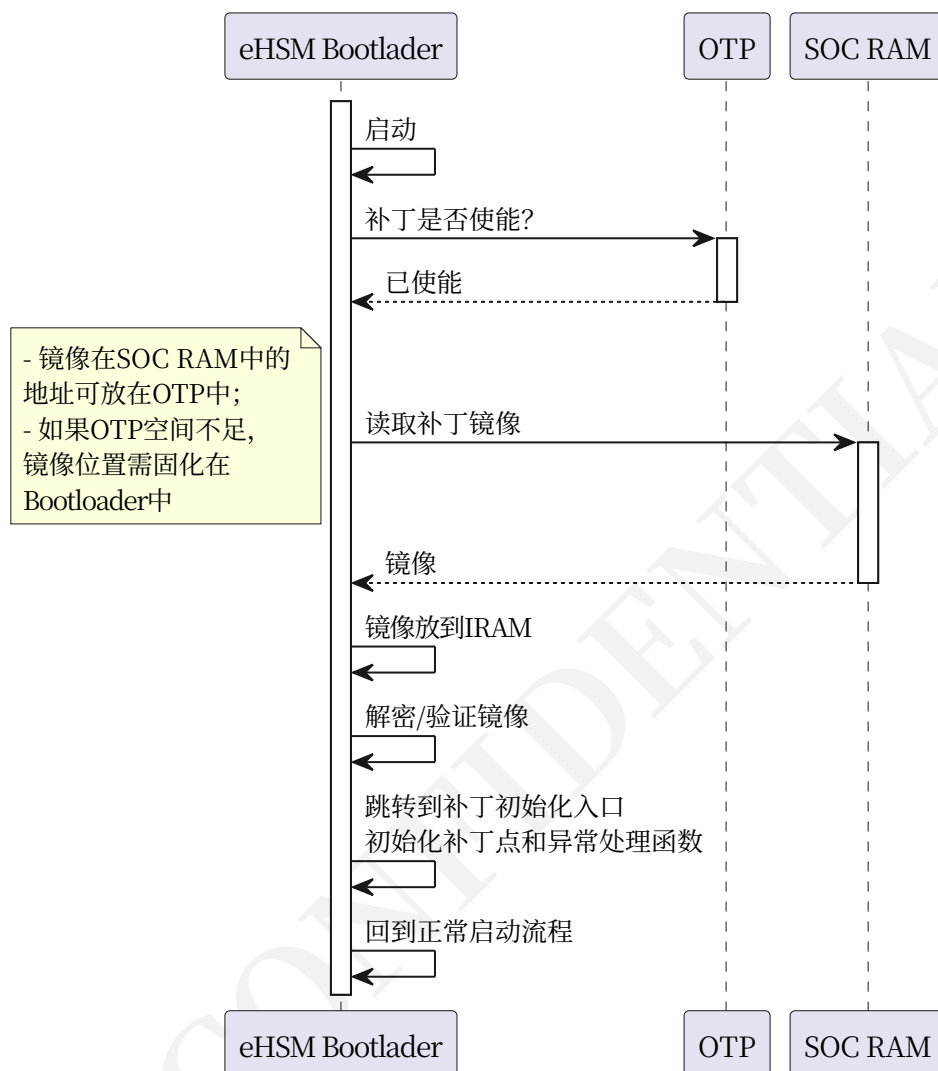


图 16 补丁加载流程

## 8.2.2 补丁命中

函数级的补丁，通过将函数入口地址的 32 位指令数据替换为 ECALL 指令，然后在异常处理函数中修改返回地址为补丁函数地址，来达到替换函数的目的。

下图是补丁的命中流程：

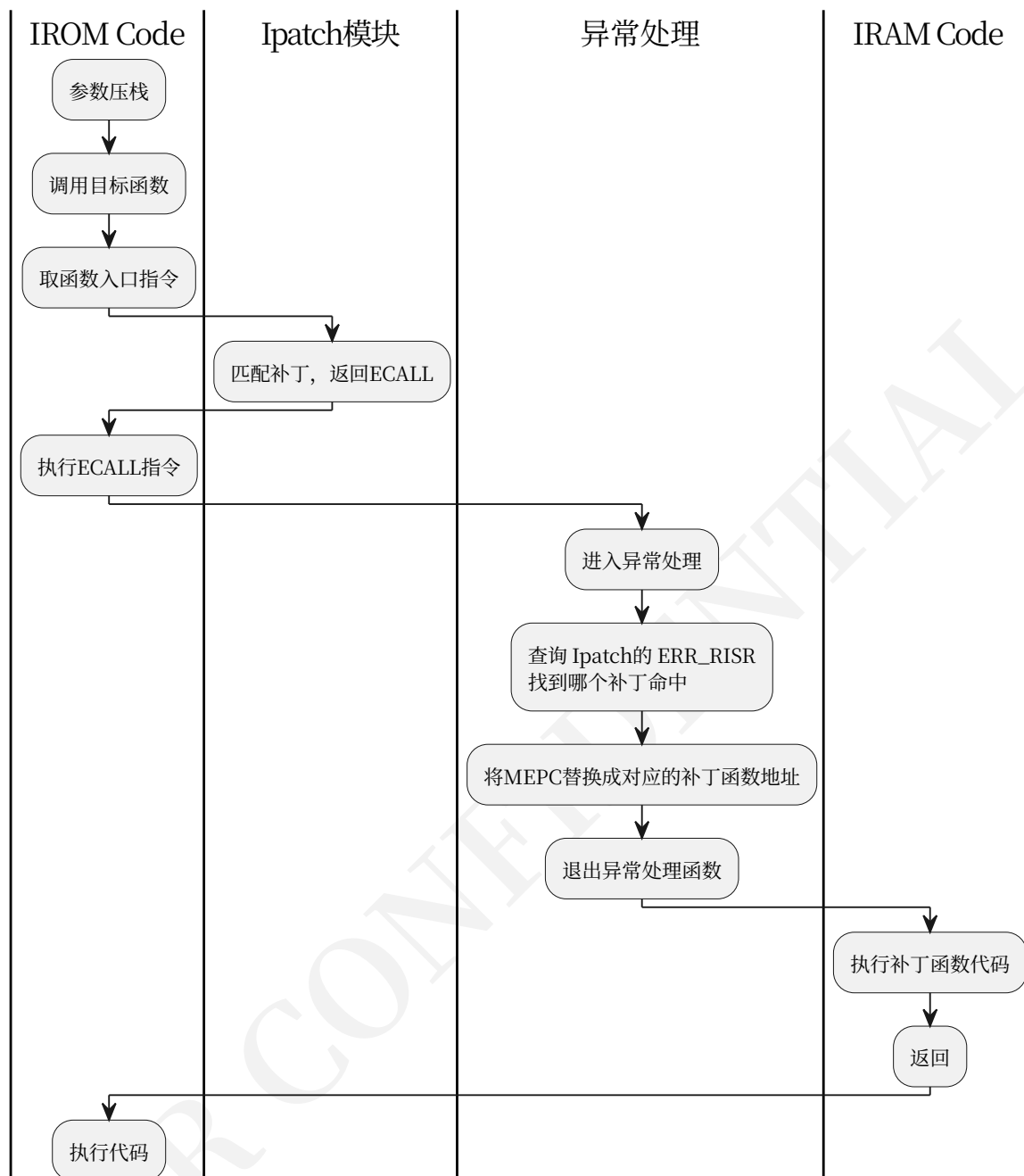


图 17 补丁命中流程

注：由于 ECALL 指令是 32 位，而 Ipatch 模块仅支持对 32 位对齐的地址打补丁，因此，如果函数地址不是 32 位对齐时，需要消耗 2 个补丁点来替换函数的前 32 位指令为 ECALL。

注 2：函数级补丁是最常的补丁方式，由于 Ipatch 本质上是对 ROM 数据进行替换，因此理论上可以将任何地址的指令替换为其它指令，不仅限于函数入口。

### 8.2.3 Patch 格式

加载时使用[安全启动 6](#)相同的镜像格式，patch 本身的数据在镜像的 code 区域。对于 code 区域的格式，没有严格的要求，由于 Bootloader 调用 patch 的 `init_entry` 进行所有的 patch 初始化工作，因此保证 `init_entry` 的入口地址在 code 区域的起始地址即可。

升级时使用[安全固件升级 7](#) 相同的镜像格式。

Patch 需要包含以下信息：

1. `init_entry`函数，用于 patch 的初始化，且函数入口地址必须是镜像 code 区域的起始地址
2. Patch 表，每个表项包含被打补丁的地址、替换指令、补丁函数地址。`init_entry`和异常处理函数会使用此表
3. 异常处理函数和补丁函数

#### 8.2.4 Patch 位置

Patch 在 eHSM IRAM 中的位置位于 IRAM 的尾部。例如一个 8K 的 Patch<sup>8</sup> 加载到 IRAM 后如下：

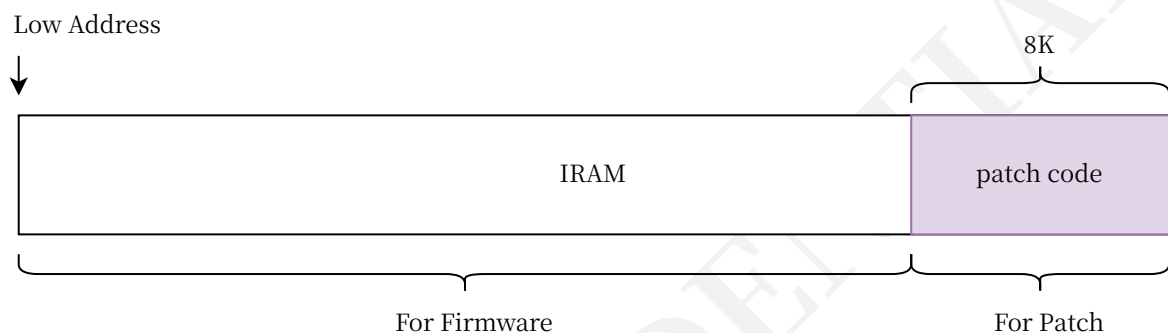


图 18 Patch 加载位置示例

注：根据 Patch 的大小和 IRAM 的地址范围，可以确定 Patch 在 IRAM 中的位置，编译 Patch 时需要指定对应的起始地址，生成 Patch 时建议将其 size 对齐到 1K。

由于 Patch 镜像是在 Bootloader 启动时加载，因此其在 SOC RAM 中固定放在一个地址，由 Bootloader 读取。此地址在由 Bootloader 的配置决定，编译时固定。

Patch 的开关在 OTP 中指定，见表 [表 5](#)。

#### 8.2.5 Patch 开发

请基于 OSR 提供的 demo 工程进行 Patch 开发，需要提供掩膜的 ROM code 对应的 bin 文件和 map 文件，请注意保存。

<sup>8</sup>这里不包含 1K 的镜像头

## 9 Mailbox 通信协议

### 9.1 硬件说明

eHSM 软件通过硬件的 Mailbox 通道交互数据信息，在硬件设计中，

- 共支持最多 16 个 mailbox 通道
- 其中下行方向（从 SOC 到 eHSM）每个 Mailbox 通道有 2 个数据寄存器 `MB_S2H_INFO[0]`、`MB_S2H_INFO[1]`，1 个状态寄存器 `MB_S2H_NOTE`
- 上行方向（从 eHSM 到 SOC）每个 Mailbox 通道有 2 个数据寄存器 `MB_H2S_INFO[0]`、`MB_H2S_INFO[1]`，1 个状态寄存器 `MB_H2S_NOTE`。

依靠硬件提供的寄存器，以及 eHSM 可以访问 SOC RAM 的特性<sup>9</sup>，eHSM 遵循相应的协议来响应上位机的命令。<sup>10</sup>

### 9.2 协议说明

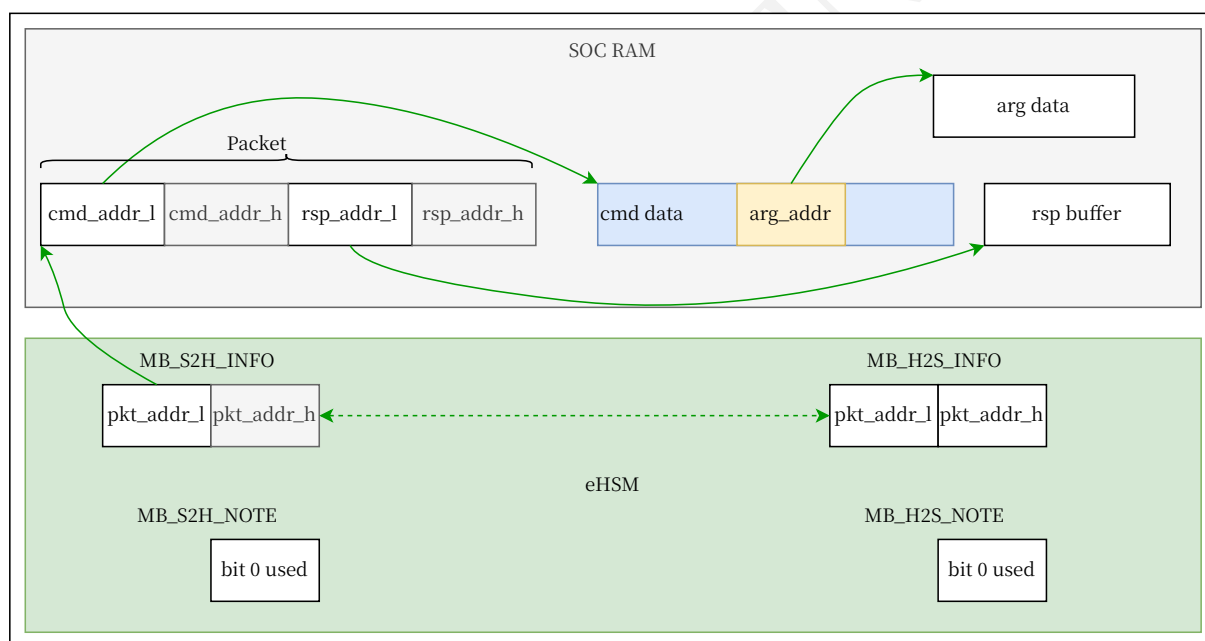


图 19 MailBox 协议数据流

说明：

- Mailbox `MB_S2H_INFO` 寄存器中存放命令 packet 的地址
- `MB_S2H_INFO` 中的 packet 地址指向 SOC RAM 中的一个 packet 结构体，结构体包含命令数据的起始地址和响应数据 buffer 的起始地址
- 设置 `MB_S2H_NOTE` 寄存器的 bit0 为 1，用于通知 eHSM 接收数据
- `MB_H2S_NOTE` 的 bit0 变为 1 时，表示命令处理完成，`MB_H2S_INFO` 中保存了 packet 地址，表明处理完成的是哪个 packet

<sup>9</sup>所有命令数据、响应 buffer 都必须存放在 eHSM 可访问的 SOC RAM 中，Mailbox 仅用于发送地址。

<sup>10</sup>硬件的 Mailbox 设计请参考 [1]。

- cmd data 按实际命令大小准备，设计 Mailbox 命令时不会超过 128 字节
- rsp buffer 大小为 20 字节，eHSM 返回的数据不会超过此大小

注：

- 图中 RAM 数据和寄存器都是左边为低地址，右边为高地址
- 如果 SOC RAM 是 32 位地址，则对应的 addr\_h 填 0
- 图中的数据都是小端字节序

协议的基本流程：

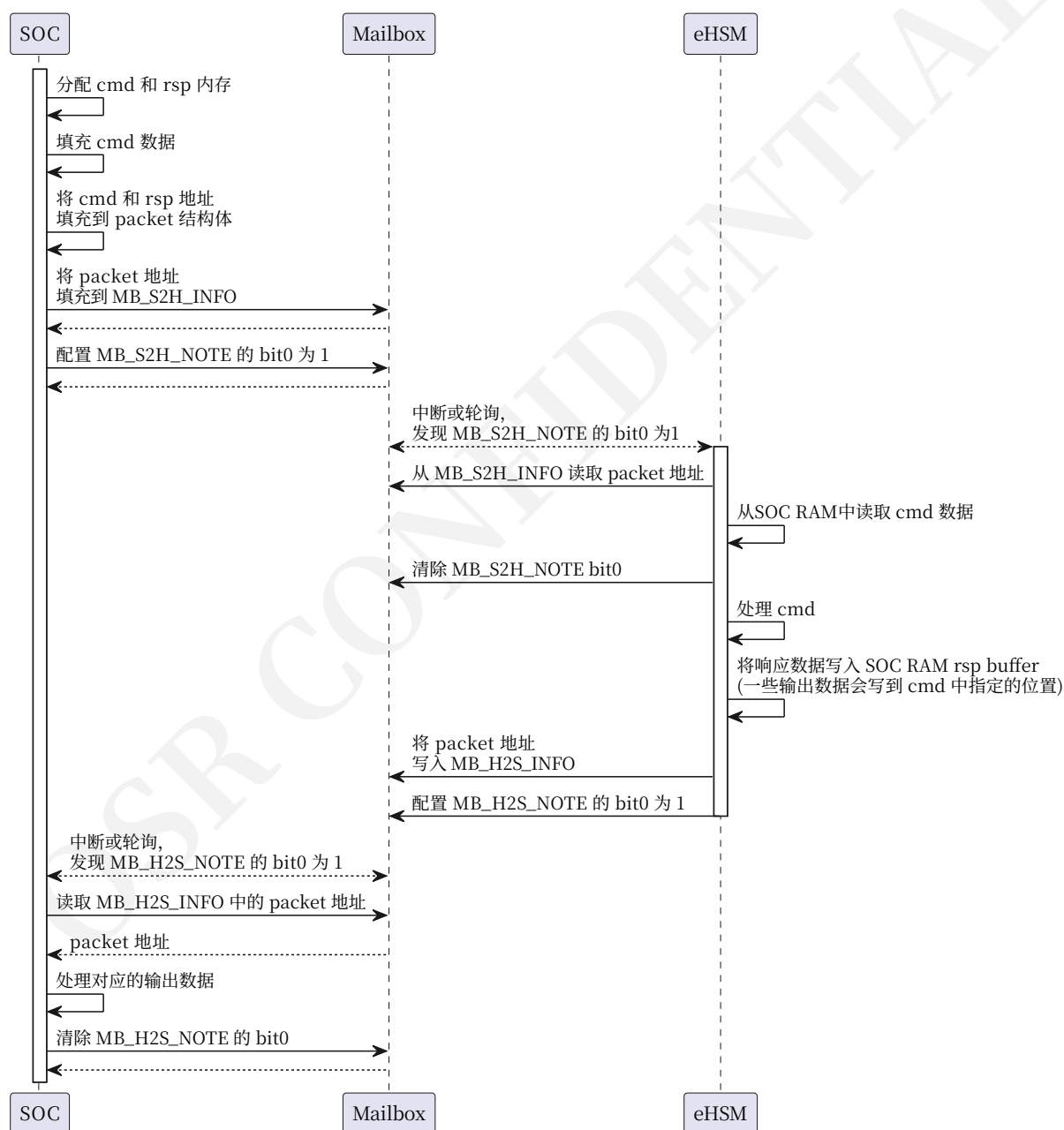


图 20 Mailbox 协议流程



## A Mailbox 命令

所有命令、结构体中的定义的超 1 字节宽的数据，均为小端存储，另有说明的除外。

### A.1 全局定义

算法的处理模式。

表 13 枚举 process\_mode 定义

| 枚举名称     | 值 | 说明           |
|----------|---|--------------|
| START    | 1 | 三段式处理的开始阶段   |
| UPDATE   | 2 | 三段式处理的更新数据阶段 |
| FINISH   | 4 | 三段式处理的结束阶段   |
| ONE-PASS | 7 | 一次性处理        |

调试身份验证挑战类型。

表 14 枚举 debug\_auth\_challenge\_type 定义

| 枚举名称                           | 值 | 说明          |
|--------------------------------|---|-------------|
| EHSM_CHALLENGE_TYPE_EHSM_DEBUG | 1 | eHSM 调试身份验证 |
| EHSM_CHALLENGE_TYPE_SOC_DEBUG  | 3 | SOC 调试身份验证  |
| EHSM_CHALLENGE_TYPE_USER_AUTH  | 4 | 保留          |
| EHSM_CHALLENGE_TYPE_FW_AUTH    | 5 | 固件身份验证      |

### A.2 BootLoader 功能

BootLoader 提供的功能命令。

用于获取 eHSM 版本信息的数据结构

表 15 结构体 ehsm\_version 定义

| 位置    | 字段名称            | 说明                                                                                             |
|-------|-----------------|------------------------------------------------------------------------------------------------|
| 0-0   | type            | 固件类型 选项： <ul style="list-style-type: none"><li>• 0: BootLoader</li><li>• 1: Firmware</li></ul> |
| 1-1   | ver_major       | 主版本号。                                                                                          |
| 2-2   | ver_minor       | 次版本号。                                                                                          |
| 3-3   | ver_patch       | 补丁版本号。                                                                                         |
| 4-11  | ver_pre_release | 预发布版本字符串，例如 <code>alpha.1</code> ， <code>rc3</code> 。对于正式版本，这是一个空字符串。                          |
| 12-19 | reserved0       | -                                                                                              |
| 20-23 | pke_engine_ver  | PKE 引擎版本号。                                                                                     |
| 24-27 | pke_lib_ver     | PKE 库版本号。格式: 0x23080301 表示 2023-08-03 版本 1。                                                    |
| 28-31 | ske_engine_ver  | SKE 引擎版本号。                                                                                     |

| 位置     | 字段名称            | 说明                                                                    |
|--------|-----------------|-----------------------------------------------------------------------|
| 32-35  | ske_lib_ver     | SKE 库版本号。格式: 同 pke_lib_ver。                                           |
| 36-39  | hash_engine_ver | HASH 引擎版本号。                                                           |
| 40-43  | hash_lib_ver    | HASH 库版本号。格式: 同 pke_lib_ver。                                          |
| 44-47  | trng_engine_ver | TRNG 引擎版本号。                                                           |
| 48-51  | trng_lib_ver    | TRNG 库版本号。格式: 同 pke_lib_ver。                                          |
| 52-55  | hw_ver          | 硬件版本号, 参见 OSR_eHSM_HP_Technical_Reference_Manual_xxx.pdf 中的 SYS_VER1。 |
| 56-71  | uid             | 从 OTP 读取的 16 字节 UID 值。                                                |
| 72-127 | reserved1       | -                                                                     |

### A.2.1 bl\_read\_ver

读取版本命令。

表 16 结构体 CMD-BL-READ-VER 定义

| 位置    | 字段名称       | 说明                        |
|-------|------------|---------------------------|
| 0-1   | cmd_id     | 0xff10                    |
| 2-3   | cmd_id_inv | cmd_id 取反                 |
| 4-11  | reserved0  | -                         |
| 12-19 | ver_addr   | 版本信息缓冲区的 host 地址。参考 表 15。 |
| 20-23 | ver_size   | 版本缓冲区的大小。                 |

响应:

表 17 结构体 RSP-BL-READ-VER 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败。 |

### A.2.2 bl\_otp\_read

读取 OTP 命令。

表 18 结构体 CMD-BL-OTP-READ 定义

| 位置    | 字段名称          | 说明                         |
|-------|---------------|----------------------------|
| 0-1   | cmd_id        | 0xff02                     |
| 2-3   | cmd_id_inv    | cmd_id 取反                  |
| 4-11  | reserved0     | -                          |
| 12-19 | ehsm_src_addr | eHSM 内部的 OTP 地址, 数据从此地址读取。 |
| 20-23 | size          | 要读取的数据的字节大小。               |
| 24-31 | host_dst_addr | host 地址, 数据将写入此地址。         |

此命令可用于读取 OTP 中的数据, 如生命周期、UID、控制字段、OTP Patch 配置表、传感器响应配置和系统密钥。此命令只能在 TEST\_MODE 和 DEVELOP\_MODE 下执行; MANUFACTURE\_MODE、

USER\_MODE、DEBUG\_MODE、DESTROY\_MODE和UNDEFINE\_MODE下禁止读取, 包括OTP Patch 配置表区域。

响应:

表 19 结构体 RSP-BL-OTP-READ 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败. |

### A.2.3 bl\_otp\_write

写入 OTP 命令。

表 20 结构体 CMD-BL-OTP-WRITE 定义

| 位置    | 字段名称          | 说明                                 |
|-------|---------------|------------------------------------|
| 0-1   | cmd_id        | 0xff01                             |
| 2-3   | cmd_id_inv    | cmd_id 取反                          |
| 4-11  | reserved0     | -                                  |
| 12-19 | ehsm_dst_addr | eHSM 地址, 数据将写入此地址。请注意, OTP 只能写入一次。 |
| 20-23 | size          | 要写入的数据的字节大小。                       |
| 24-31 | host_src_addr | host 地址, 数据从此地址读取。                 |

此命令可用于将数据写入 OTP, 如生命周期、UID、控制字段、OTP Patch 配置表、传感器响应配置和系统密钥。此命令只能在 TEST\_MODE 和 DEVELOP\_MODE 下执行; MANUFACTURE\_MODE、USER\_MODE、DEBUG\_MODE、DESTROY\_MODE 和 UNDEFINE\_MODE 下禁止写入, 包括 OTP Patch 配置表区域。OTP Patch 配置表写入后需要重新上电或复位, 硬件才会重新加载并生效。

响应:

表 21 结构体 RSP-BL-OTP-WRITE 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败. |

### A.2.4 bl\_reg\_rd

CFG 数据读取命令。

表 22 结构体 CMD-BL-REG-RD 定义

| 位置    | 字段名称          | 说明                        |
|-------|---------------|---------------------------|
| 0-1   | cmd_id        | 0xff24                    |
| 2-3   | cmd_id_inv    | cmd_id 取反                 |
| 4-11  | reserved0     | -                         |
| 12-19 | ehsm_src_addr | eHSM 地址, 数据将从此地址读取。       |
| 20-23 | size          | 要读取的数据的字节大小。固件无需验证此值的有效性。 |

| 位置    | 字段名称          | 说明                           |
|-------|---------------|------------------------------|
| 24-31 | host_dst_addr | host 地址，数据将写入此地址。缓冲区应有足够的大小。 |

此命令可用于读取 0x33400000 至 0x334FFFFFF 地址范围内的数据。此命令只能在 TEST\_MODE 和 DEVELOP\_MODE 下执行。

响应：

表 23 结构体 RSP-BL-REG-RD 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败. |

## A.2.5 bl\_reg\_wr

CFG 数据写入命令。

表 24 结构体 CMD-BL-REG-WR 定义

| 位置    | 字段名称          | 说明                 |
|-------|---------------|--------------------|
| 0-1   | cmd_id        | 0xff23             |
| 2-3   | cmd_id_inv    | cmd_id 取反          |
| 4-11  | reserved0     | -                  |
| 12-19 | ehsm_dst_addr | eHSM 地址，数据将被写入此地址。 |
| 20-23 | size          | 要写入的数据的字节大小。       |
| 24-31 | host_src_addr | host 地址，数据将从此地址读取。 |

此命令可用于读取 0x33400000 至 0x334FFFFFF 地址范围内的数据。此命令只能在 TEST\_MODE 和 DEVELOP\_MODE 下执行。

响应：

表 25 结构体 RSP-BL-REG-WR 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败. |

## A.2.6 bl\_fw\_upgrade

固件镜像升级命令。

表 26 结构体 CMD-BL-FW-UPGRADE 定义

| 位置    | 字段名称         | 说明            |
|-------|--------------|---------------|
| 0-1   | cmd_id       | 0xff05        |
| 2-3   | cmd_id_inv   | cmd_id 取反     |
| 4-11  | reserved0    | -             |
| 12-12 | process_mode | 处理模式。参考 表 13。 |

| 位置    | 字段名称         | 说明                                                                                            |
|-------|--------------|-----------------------------------------------------------------------------------------------|
| 13-31 | reserved1    | -                                                                                             |
| 32-39 | image_addr   | 镜像地址。ONE-PASS 时，它是整个镜像的 host 地址，START 时为镜像头部，UPDATE 时为镜像加密部分地址，或者 FINISH 时为镜像最后一部分加密内容（可能为空）。 |
| 40-43 | image_size   | 升级镜像的大小（以字节为单位）。                                                                              |
| 44-51 | storage_addr | 镜像存储地址它是镜像存储的 host 地址。                                                                        |

用于引导程序中的镜像升级。

响应：

表 27 结构体 RSP-BL-FW-UPGRADE 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败。 |

## A.2.7 bl\_verify\_image

镜像校验命令。

表 28 结构体 CMD-BL-VERIFY-IMAGE 定义

| 位置    | 字段名称              | 说明                                                                                                                                          |
|-------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 0-1   | cmd_id            | 0xff06                                                                                                                                      |
| 2-3   | cmd_id_inv        | cmd_id 取反                                                                                                                                   |
| 4-13  | reserved0         | -                                                                                                                                           |
| 14-14 | check_version     | 是否检查版本计数器。选项： <ul style="list-style-type: none"> <li>• 0x00: OFF</li> <li>• 0x01: ON</li> <li>• 其他值保留</li> </ul>                            |
| 15-15 | boot_after_verify | 验证成功后是否启动 eHSM FW。选项： <ul style="list-style-type: none"> <li>• 0x01: ON</li> <li>• 0x00: OFF</li> <li>• 其他值保留 仅对 eHSM FW 镜像有效。</li> </ul>   |
| 16-23 | image_addr        | 如果 code_addr 为 0，则是完整镜像的 host 地址；否则是镜像头的 host 地址。                                                                                           |
| 24-27 | image_size        | 镜像的大小（以字节为单位），包括镜像头和 code 区。                                                                                                                |
| 28-35 | image_out_addr    | 镜像解密后（如果镜像中是明文，则复制）存储到的 host 地址。仅对 SOC 镜像有效，可以指向原始镜像地址。                                                                                     |
| 36-43 | code_addr         | 代码的 host 地址，如果 code_addr 为 0，则 code 存储在 image_addr 指向的镜像头后面。                                                                                |
| 44-44 | only_copy_code    | 是否仅将代码复制到 image_out_addr。选项： <ul style="list-style-type: none"> <li>• 0x01: YES</li> <li>• 0x00: NO</li> <li>• 其他值保留 仅对 SOC 镜像有效</li> </ul> |

此命令用于解密并验证 eHSM 或 SOC 镜像。

响应:

表 29 结构体 RSP-BL-VERIFY-IMAGE 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败. |

## A.2.8 bl\_get\_random\_key

生成随机密钥命令。

表 30 结构体 CMD-BL-GET-RANDOM-KEY 定义

| 位置    | 字段名称       | 说明                                                                                                                                                                                                                                                                                                                                                                                  |
|-------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0-1   | cmd_id     | 0xff08                                                                                                                                                                                                                                                                                                                                                                              |
| 2-3   | cmd_id_inv | cmd_id 取反                                                                                                                                                                                                                                                                                                                                                                           |
| 4-11  | reserved0  | -                                                                                                                                                                                                                                                                                                                                                                                   |
| 12-12 | key_level  | eHSM 或 SOC 密钥 选项:<br><ul style="list-style-type: none"> <li>1: 生成 CHIP_ROOT_KEY 或者随机密钥使用 CHIP_ROOT_KEY 加密。eHSM 密钥生成仅支持 TEST_MODE 和 DEVELOP_MODE</li> <li>2: 随机密钥使用 DEVICE_ROOT_KEY 加密。SOC 密钥生成支持 TEST_MODE、DEVELOP_MODE 和 MANUFACTURE_MODE</li> <li>0xFF: 随机密钥是 DEVICE_ROOT_KEY, 并使用 CHIP_ROOT_KEY 加密。随机 DEVICE_ROOT_KEY 的生成支持 TEST_MODE、DEVELOP_MODE 和 MANUFACTURE_MODE</li> </ul> |
| 13-13 | key_type   | 密钥类型。选项:<br><ul style="list-style-type: none"> <li>1: 对称密钥</li> <li>2: SM2 私钥</li> <li>4: SECP256R1 私钥 对于对称密钥, 输出是随机的 32 字节密钥。对于 SM2 或 SECP256R1, 输出是随机生成的有效私钥。</li> </ul>                                                                                                                                                                                                          |
| 14-39 | reserved1  | -                                                                                                                                                                                                                                                                                                                                                                                   |
| 40-47 | key_addr   | 输出加密随机密钥和 4 字节 CRC32 值的 host 地址。                                                                                                                                                                                                                                                                                                                                                    |

响应:

表 31 结构体 RSP-BL-GET-RANDOM-KEY 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败. |

## A.2.9 bl\_encrypt\_key

加密密钥命令。

表 32 结构体 CMD-BL-ENCRYPT-KEY 定义

| 位置  | 字段名称   | 说明     |
|-----|--------|--------|
| 0-1 | cmd_id | 0xff09 |

| 位置    | 字段名称        | 说明                                                                                                                                                                                                                                                                                                                                                      |
|-------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2-3   | cmd_id_inv  | cmd_id 取反                                                                                                                                                                                                                                                                                                                                               |
| 4-11  | reserved0   | -                                                                                                                                                                                                                                                                                                                                                       |
| 12-12 | key_level   | eHSM 或 SOC 密钥 选项： <ul style="list-style-type: none"> <li>• 1: 输入密钥使用 CHIP_ROOT_KEY 加密。eHSM 密钥生成仅支持 TEST_MODE 和 DEVELOP_MODE。</li> <li>• 2: 输入密钥使用 DEVICE_ROOT_KEY 加密。SOC 密钥生成支持 TEST_MODE、DEVELOP_MODE 和 MANUFACTURE_MODE。</li> <li>• 0xFF: 输入 密 钥 是 DEVICE_ROOT_KEY， 并 使 用 CHIP_ROOT_KEY 加 密 。DEVICE_ROOT_KEY 的加密支持 TEST_MODE、DEVELOP_MODE。</li> </ul> |
| 13-31 | reserved1   | -                                                                                                                                                                                                                                                                                                                                                       |
| 32-39 | input_addr  | 输入对称密钥或非对称密钥的私钥（只支持 SM2 或 SECP256R1）或公钥哈希的 host 地址。32 字节密钥（或哈希）+ 4 字节密钥（或哈希）的 CRC 值，用 0x00 填充至 48 字节，并使用 RTL（eHSM/SOC）KEK 加密的结果。                                                                                                                                                                                                                        |
| 40-43 | input_size  | 输入密钥或 HASH 的大小，必须为 48 字节。                                                                                                                                                                                                                                                                                                                               |
| 44-51 | output_addr | 存储加密密钥或 HASH 以及 32 字节大小的加密密钥的 4 字节 CRC32 值的 host 地址。                                                                                                                                                                                                                                                                                                    |

对输入的密钥密文使用 KEK 解密，并使用 ROOT KEY 加密，然后输出。

响应：

表 33 结构体 RSP-BL-ENCRYPT-KEY 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败。 |

## A.2.10 bl\_get\_challenge

获取挑战值命令。

表 34 结构体 CMD-BL-GET-CHALLENGE 定义

| 位置    | 字段名称       | 说明                                                                                    |
|-------|------------|---------------------------------------------------------------------------------------|
| 0-1   | cmd_id     | 0xff03                                                                                |
| 2-3   | cmd_id_inv | cmd_id 取反                                                                             |
| 4-11  | reserved0  | -                                                                                     |
| 12-12 | type       | 获取挑战值的类型。参考 表 14。                                                                     |
| 13-47 | reserved1  | -                                                                                     |
| 48-55 | addr       | 存储挑战值的 host 地址。对于 eHSM 调试认证和 SOC 调试认证，应有 32 字节的随机数据和 16 字节的 UID；对于固件认证，应有 16 字节的随机数据。 |

响应：

表 35 结构体 RSP-BL-GET-CHALLENGE 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败。 |

### A.2.11 bl\_debug\_auth

调试认证命令。

表 36 结构体 CMD-BL-DEBUG-AUTH 定义

| 位置    | 字段名称                | 说明                                                                                                                                                                                                                                            |
|-------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0-1   | cmd_id              | 0xff04                                                                                                                                                                                                                                        |
| 2-3   | cmd_id_inv          | cmd_id 取反                                                                                                                                                                                                                                     |
| 4-11  | reserved0           | -                                                                                                                                                                                                                                             |
| 12-12 | type                | 之前获取挑战值的类型。参考 表 14。                                                                                                                                                                                                                           |
| 13-13 | alg                 | 验证算法。选项： <ul style="list-style-type: none"> <li>• 0x01: SM3-SM2</li> <li>• 0x02: SHA256-SECP256R1</li> <li>• 0x03: SM4-CMAC</li> <li>• 0x04: AES128-CMAC</li> <li>• 0x05: SHA256_RSA</li> </ul>                                               |
| 14-23 | reserved1           | -                                                                                                                                                                                                                                             |
| 24-31 | sign_addr           | 签名的 host 地址。                                                                                                                                                                                                                                  |
| 32-35 | sign_size           | 签名的字节大小。对于 eHSM/SOC 调试认证，算法为 SM3-SM2 和 SHA256-ECDSA-SECP256R1 时，值应为 64 字节；算法为 SHA256-RSA2048-PSS 时，值应为 256 字节；算法为 SHA256-RSA3072-PSS 时，值应为 384 字节。                                                                                            |
| 36-43 | pub_addr            | 签名使用的公钥的 host 地址。仅对 eHSM/SOC 调试认证有效。                                                                                                                                                                                                          |
| 44-47 | pub_size            | 签名者公钥的字节大小。仅对 eHSM/SOC 调试认证有效。对于 SM3-SM2 算法，大小应为 65 字节（包括未压缩标志' 0x04'）。对于 SHA256-ECDSA-SECP256R1，大小为 64 字节；对于 SHA256-RSA2048-PSS，大小为 320 字节（64 字节的 RSA(e) 和 256 字节的 RSA(n)）；对于 SHA256-RSA3072-PSS，大小为 448 字节（64 字节的 RSA(e) 和 384 字节的 RSA(n)）。 |
| 48-55 | soc_dbg_bitmap_addr | SOC 调试认证位图数据的 host 地址。仅对 SOC 调试认证有效。前 4 个字对应寄存器 SYS_WR_SOC_DBG_EN_128B0 至 SYS_WR_SOC_DBG_EN_128B3（为 1 的位表示打开对应端口，为 0 的位表示不改变对应端口状态）。最后一个字对应 SYS_WR_SOC_DBG_EN（仅 bit0 有效）。                                                                     |
| 56-59 | soc_dbg_bitmap_size | SOC 调试身份验证位图数据的大小。固定长度（5 个字）。                                                                                                                                                                                                                 |

响应：

表 37 结构体 RSP-BL-DEBUG-AUTH 定义

| 位置  | 字段名称     | 说明                          |
|-----|----------|-----------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功，其它值表示失败。 |

### A.2.12 bl\_close\_debug

关闭调试功能。

表 38 结构体 CMD-BL-CLOSE-DEBUG 定义



| 位置    | 字段名称                | 说明                                                                                                                                       |
|-------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 0-1   | cmd_id              | 0xff0e                                                                                                                                   |
| 2-3   | cmd_id_inv          | cmd_id 取反                                                                                                                                |
| 4-11  | reserved0           | -                                                                                                                                        |
| 12-12 | type                | 关闭调试的类型。参考 表 14。                                                                                                                         |
| 13-15 | reserved1           | -                                                                                                                                        |
| 16-23 | soc_dbg_bitmap_addr | SoC 调试认证位图数据的 host 地址。仅对 SoC 调试认证有效。大小为 5 个字，前 4 个字表示是否按位关闭相应的硬件调试端口（为 1 的位表示关闭对应端口，为 0 的位表示不改变对应端口状态）。最后一个字仅最低位有效（表示是否关闭 129 位的硬件调试端口）。 |
| 24-27 | soc_dbg_bitmap_size | SOC 调试身份验证位图数据的大小。固定长度（5 个字）。                                                                                                            |

响应：

表 39 结构体 RSP-BL-CLOSE-DEBUG 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败。 |

### A.2.13 bl\_set\_uart\_baudrate

设置 UART 波特率的命令。

表 40 结构体 CMD-BL-SET-UART-BAUDRATE 定义

| 位置  | 字段名称       | 说明                                    |
|-----|------------|---------------------------------------|
| 0-1 | cmd_id     | 0x0f04                                |
| 2-3 | cmd_id_inv | cmd_id 取反                             |
| 4-7 | baud_div   | UART 的频率分频值，指 CPU 频率值除以 UART 波特率值的结果。 |

响应：

表 41 结构体 RSP-BL-SET-UART-BAUDRATE 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败。 |

### A.2.14 bl\_self\_test

自检命令。将测试所有支持的算法类型。

表 42 结构体 CMD-BL-SELF-TEST 定义

| 位置  | 字段名称       | 说明        |
|-----|------------|-----------|
| 0-1 | cmd_id     | 0x0020    |
| 2-3 | cmd_id_inv | cmd_id 取反 |

响应：

表 43 结构体 RSP-BL-SELF-TEST 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败. |

### A.2.15 bl\_read\_self\_test\_result

读取自检结果的命令。

表 44 结构体 CMD-BL-READ-SELF-TEST-RESULT 定义

| 位置    | 字段名称        | 说明                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0-1   | cmd_id      | 0xff11                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 2-3   | cmd_id_inv  | cmd_id 取反                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 4-11  | reserved0   | -                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 12-19 | result_addr | 自检结果存储的 host 地址。结果包含两个字, 第一个字表示每个算法的结果, 第二个字表示已测试的算法。由对应的位标志组合而成。支持的算法类型: <ul style="list-style-type: none"><li>• 0x1: SM4 ECB 模式</li><li>• 0x2: SM4 CBC 模式</li><li>• 0x4: SM4 CFB 模式</li><li>• 0x8: SM4 OFB 模式</li><li>• 0x10: SM4 CTR 模式</li><li>• 0x20: SM2 加密</li><li>• 0x40: SM2 签名验证</li><li>• 0x80: SM2 密钥交换</li><li>• 0x100: SM3</li><li>• 0x200: DES</li><li>• 0x400: TDES</li><li>• 0x800: AES</li><li>• 0x1000: RSA</li><li>• 0x2000: ECC</li><li>• 0x4000: MD5</li><li>• 0x8000: SHA1</li><li>• 0x10000: SHA2</li><li>• 0x20000: SHA3</li><li>• 0x40000: TRNG</li></ul> |

响应:

表 45 结构体 RSP-BL-READ-SELF-TEST-RESULT 定义

| 位置  | 字段名称     | 说明                           |
|-----|----------|------------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功, 其它值表示失败. |

## A.2.16 bl\_fw\_auth

固件认证命令。

表 46 结构体 CMD-BL-FW-AUTH 定义

| 位置    | 字段名称       | 说明                                                                                                                                                                                             |
|-------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0-1   | cmd_id     | 0xff12                                                                                                                                                                                         |
| 2-3   | cmd_id_inv | cmd_id 取反                                                                                                                                                                                      |
| 4-11  | reserved0  | -                                                                                                                                                                                              |
| 12-12 | type       | 认证类型。选项： <ul style="list-style-type: none"> <li>• 0x01: 删除给定的用户密钥</li> <li>• 0x02: 修改 eHSM 的版本计数器</li> <li>• 0x03: 进入销毁模式，删除所有密钥 不同类型使用不同的密钥。类型 1 使用用户认证密钥，类型 2 和类型 3 使用 eHSM 认证密钥。</li> </ul> |
| 13-15 | reserved1  | -                                                                                                                                                                                              |
| 16-31 | arg        | 当类型为 1 时，arg 表示用户密钥 ID；当类型为 2 时，表示新的版本计数器；否则忽略。                                                                                                                                                |
| 32-63 | auth_data  | 认证数据。认证数据，表示随机数据 Ra  Rb 的 SM4-CBC 加密结果，Ra 由 bl_get_challenge 命令生成，Rb 由主机生成。Ra 和 Rb 均为 16 字节。                                                                                                   |
| 64-71 | out_addr   | OTP 输出 buffer，可以为 NULL，当不为 NULL 时，应当至少有 720 字节空间。                                                                                                                                              |

需要先执行 bl\_get\_challenge 命令。

响应：

表 47 结构体 RSP-BL-FW-AUTH 定义

| 位置  | 字段名称     | 说明                          |
|-----|----------|-----------------------------|
| 0-3 | ret_code | 0x0000A55AU 表示操作成功，其它值表示失败。 |

## B 错误码

下表定义了 Mailbox 命令返回的错误码编码。

表 48 错误码定义

| Name                              | Value |
|-----------------------------------|-------|
| EHSM_ERR_PARAM_ERROR              | 1     |
| EHSM_ERR_QUEUE_FULL               | 2     |
| EHSM_ERR_NOT_INIT                 | 3     |
| EHSM_ERR_REPEATED_INIT            | 4     |
| EHSM_ERR_FREE_REPEATED            | 5     |
| EHSM_ERR_QUEUE_EMPTY              | 6     |
| EHSM_ERR_CMDPOOL_EMPTY            | 7     |
| EHSM_ERR_INVALID_ADDRESS          | 8     |
| EHSM_ERR_WONG_IRQ_NUM             | 9     |
| EHSM_ERR_NOT_SUPPORT              | 10    |
| EHSM_ERR_INVALID_HANDLE           | 11    |
| EHSM_ERR_MISMATCH_KEY_USAGE       | 12    |
| EHSM_ERR_NOT_ALIGNED              | 13    |
| EHSM_ERR_DATA_NOT_EMPTY           | 14    |
| EHSM_ERR_INIT_DEV_FAIL            | 15    |
| EHSM_ERR_WRITE_DEV_FAIL           | 16    |
| EHSM_ERR_READ_DEV_FAIL            | 17    |
| EHSM_ERR_ERASE_DEV_FAIL           | 18    |
| EHSM_ERR_CTX_OVERFLOW             | 19    |
| EHSM_ERR_OUTPUT_OVERFLOW          | 20    |
| EHSM_ERR_HMAC_K_BUF_OCCUPIED      | 21    |
| EHSM_ERR_HMAC_INVALID_K_IDX       | 22    |
| EHSM_ERR_REMAP_FAILED             | 23    |
| EHSM_ERR_REMAP_ADDRESS_OVERFLOW   | 24    |
| EHSM_ERR_MAC_LEN_WRONG_FORMAT     | 25    |
| EHSM_ERR_MAC_VRY_FAILED           | 26    |
| EHSM_ERR_INVALID_DIR              | 27    |
| EHSM_ERR_OUT_OF_MEM               | 28    |
| EHSM_ERR_DBG_BUF_OVERFLOW         | 29    |
| EHSM_ERR_DBG_CMD_INCOMPLETE       | 30    |
| EHSM_ERR_DBG_PROTO_ERROR          | 31    |
| EHSM_ERR_INVALID_CHANNEL          | 32    |
| EHSM_ERR_NO_CMD_EXEC_PERM         | 33    |
| EHSM_ERR_INVALID_CMD              | 34    |
| EHSM_ERR_SM2_DATA_BUF_OVERFLOW    | 35    |
| EHSM_ERR_SM2_SIGNATURE_GEN_FAILED | 36    |

| Name                                | Value |
|-------------------------------------|-------|
| EHSM_ERR_SM2_SIGNATURE_VRY_FAILED   | 37    |
| EHSM_ERR_RSA_CALCULATE_FAILED       | 38    |
| EHSM_ERR_WRONG_SZ_OF_SIGNATURE      | 39    |
| EHSM_ERR_LIMIT_OF_AUTHORITY         | 40    |
| EHSM_ERR_VERIFY_KEY_INTEGRITY       | 43    |
| EHSM_ERR_EXEC_CRYPTO_LIB            | 44    |
| EHSM_ERR_MISMATCH_KEY_PERMISSION    | 45    |
| EHSM_ERR_NOT_EXIST_KEY_PART         | 46    |
| EHSM_ERR_NOT_EXIST_KEY              | 47    |
| EHSM_ERR_INVALID_KEY_ID             | 48    |
| EHSM_ERR_INVALID_ALGORITHM          | 49    |
| EHSM_ERR_DH_KEY_TOO_LEN             | 50    |
| EHSM_ERR_NOT_ALLOWED_CREATION_KEY   | 51    |
| EHSM_ERR_NOT_EMPTY_KEY_HANDLE       | 52    |
| EHSM_ERR_IV_OVERFLOW                | 53    |
| EHSM_ERR_KEY_SIGNATURE_SZ_TOO_SHORT | 54    |
| EHSM_ERR_XTS_WRONG_DATA_LENGTH      | 55    |
| EHSM_ERR_INVALID_CIPHER_MODE        | 57    |
| EHSM_ERR_INVALID_PADDING            | 58    |
| EHSM_ERR_NOT_FREE_SPACE             | 59    |
| EHSM_ERR_NVM_DATA_CHECKSUM          | 60    |
| EHSM_ERR_NVM_WRITE_DATA             | 61    |
| EHSM_ERR_NVM_SYNC_DATA              | 62    |
| EHSM_ERR_ECDSA_SIGNATURE_GEN_FAILED | 63    |
| EHSM_ERR_ECDSA_SIGNATURE_VRY_FAILED | 64    |
| EHSM_ERR_SM2_CIPHER_ENC_FAILED      | 65    |
| EHSM_ERR_SM2_CIPHER_DEC_FAILED      | 66    |
| EHSM_ERR_INPUT_OVERFLOW             | 67    |
| EHSM_ERR_RSA_SIGNATURE_GEN_FAILED   | 68    |
| EHSM_ERR_RSA_SIGNATURE_VRY_FAILED   | 69    |
| EHSM_ERR_KEY_NOT_FOUND              | 70    |
| EHSM_ERR_WRONG_IV_SIZE              | 71    |
| EHSM_ERR_NVM_ENCRYPTION_KEY_DATA    | 72    |
| EHSM_ERR_NVM_SIGNATURE_KEY_DATA     | 73    |
| EHSM_ERR_GCM_TAG_VRY_FAILED         | 74    |
| EHSM_ERR_CCM_TAG_VRY_FAILED         | 75    |
| EHSM_ERR_KEY_UPDATE_ERROR           | 76    |
| EHSM_ERR_KEY_INVALID                | 77    |
| EHSM_ERR_KEY_NOT_AVAILABLE          | 78    |
| EHSM_ERR_KEY_EMPTY_ERROR            | 79    |
| EHSM_ERR_KEY_WRITE_PROTECTED        | 80    |
| EHSM_ERR_EHSM_LIFECYCLE_LIMIT       | 81    |

| Name                                 | Value |
|--------------------------------------|-------|
| EHSM_ERR_WRONG_CHALLENGE_TYPE        | 82    |
| EHSM_ERR_WRONG_DATA_LENGTH           | 83    |
| EHSM_ERR_WRONG_ALGORITHM             | 84    |
| EHSM_ERR_WRONG_KEY_TYPE              | 85    |
| EHSM_ERR_DEBUG_AUTH_PK_HASH_MISMATCH | 86    |
| EHSM_ERR_DEBUG_AUTH_FAILED           | 87    |
| EHSM_ERR_SEQUENCE                    | 88    |
| EHSM_ERR_NO_DATA_IN_MAC_FINAL        | 89    |
| EHSM_ERR_SKE_WORK_ERROR              | 256   |
| EHSM_ERR_PKE_WORK_ERROR              | 257   |
| EHSM_ERR_HASH_WORK_ERROR             | 258   |
| EHSM_ERR_TRNG_WORK_ERROR             | 259   |
| EHSM_ERR_WRONG_K_LEVEL               | 260   |
| EHSM_ERR_WRONG_AUTH_TYPE             | 261   |
| EHSM_ERR_WRONG_PUBKEY                | 262   |
| EHSM_ERR_WRONG_VERSION_COUNTER       | 263   |
| EHSM_ERR_FW_VERIFY_FAILED            | 264   |
| EHSM_ERR_WRONG_PROC_MODE             | 265   |
| EHSM_ERR_DATA_CHECK_ERROR            | 266   |
| EHSM_ERR_WRONG_FW_TYPE               | 267   |
| EHSM_ERR_WRONG_CONTEXT               | 268   |
| EHSM_ERR_INVALID_CODE_FLAG           | 269   |
| EHSM_ERR_FW_AUTH_FAILED              | 270   |
| EHSM_ERR_SELFTEST_FAILED             | 280   |
| EHSM_ERR_WRONG_KEY_SIZE              | 281   |
| EHSM_ERR_CALC_HASH                   | 282   |
| EHSM_ERR_INVALID_IMAGE_SIZE          | 283   |

## C 字符调试鉴权协议

字符调试鉴权协议所有数据使用 ASCII 字符串进行传输，可以通过串口和 Mailbox 通道传输，实现调试鉴权的功能主要包含以下几条指令：

- 获取挑战值指令
- 签名校验指令
- 关闭调试指令
- 测试指令

以下指令格式中 type 代表调试类型，其值可以为 0x31 或 0x33, 0x31 代表 eHSM 调试鉴权，0x33 代表 SOC 调试鉴权，alg 代表算法类型，其值可以为 0x31、0x32、0x33、0x34 或 0x35, 0x31 代表 SM3-SM2, 0x32 代表 SHA256-SECP256R1, 0x33 代表 SM4-CMAC, 0x34 代表 AES128-CMAC, 0x35 代表 RSA2048-SHA256 或者 RSA3072-SHA256。

RSA 签名算法填充模式为 RSA-PSS 模式，RSA2048-SHA256 对应的公钥为：64 字节 e 值+256 字节 n 值，签名长度为 256 字节，RSA3072-SHA256 对应的公钥为：64 字节 e 值+384 字节 n 值，签名长度为 384 字节。

### C.1 获取挑战值指令

请求数据格式：“REQ” + type + alg + “@”

例如：字符串“REQ11@”代表 eHSM 调试鉴权，鉴权算法为 SM3-SM2

成功响应格式：“CHA” + type + “:” + challeng in char + “@”

challeng in char：挑战值转换的 16 进制字符串，挑战值包含 32 字节随机数+16 字节 UID

例如响应数据为：

```
1| CHA3:6343F25AA6AA8E5705CBD3D6B19BEE50D4495575562B5B1F6AE415066E2228C3008E69D927E
2| 34EB857F7BD4FAE47F9820
```

代表当前为 SOC 调试鉴权，对应的 32 字节随机数为[0x63, 0x43, 0xF2, 0x5A, 0xA6, 0xAA, 0x8E, 0x57, 0x05, 0xCB, 0xD3, 0xD6, 0xB1, 0x9B, 0xEE, 0x50, 0xD4, 0x49, 0x55, 0x75, 0x56, 0x2B, 0x5B, 0x1F, 0x6A, 0xE4, 0x15, 0x06, 0x6E, 0x22, 0x28, 0xC3], 16 字节的 UID 为 [0x00, 0x8E, 0x69, 0xD9, 0x27, 0xE3, 0x4E, 0xB8, 0x57, 0xF7, 0xBD, 0x4F, 0xAE, 0x47, 0xF9, 0x82]

失败响应格式：“CHA” + type + “:get random fail”

### C.2 签名校验指令

请求数据格式：“RSP” + type + alg + “:” + publickey + “:” + signature + “@” publickey:

非对称算法的公钥的 16 进制字符串，signature: 签名值的 16 进制字符串

例如：

```
1| RSP11:09F9DF311E5421A150DD7D161E4BC5C672179FAD1833FC076BB08FF356F35020CCEA490CE2
2| 6775A52DC6EA718CC1AA600AED05FBF35E084A6632F6072DA9AD13:A8E776DCF22DE9C5E847CC321
3| 5A298D1B99884D68A43A83A8C236347287514A2B7AA77A70727BF37A3BE69982F87778AA28C72183
4| 2C64BE5139646948EA2AF6B0
```

代表 SOC 调试鉴权的签名检验指令，算法类型是 SM3-SM2，SM2 的公钥为 [0x09, 0xF9, 0xDF, 0x31, 0x1E, 0x54, 0x21, 0xA1, 0x50, 0xDD, 0x7D, 0x16, 0x1E, 0x4B, 0xC5, 0xC6, 0x72, 0x17, 0x9F, 0xAD, 0x18, 0x33, 0xFC, 0x07, 0x6B, 0xB0, 0x8F, 0xF3, 0x56, 0xF3, 0x50, 0x20, 0xCC, 0xEA, 0x49, 0x0C, 0xE2, 0x67, 0x75, 0xA5, 0x2D, 0xC6, 0xEA, 0x71, 0x8C, 0xC1, 0xAA, 0x60, 0x0A, 0xED, 0x05, 0xFB, 0xF3, 0x5E, 0x08, 0x4A, 0x66, 0x32, 0xF6, 0x07, 0x2D, 0xA9, 0xAD, 0x13], 签名值为 [0xA8, 0xE7, 0x76, 0xDC, 0xF2, 0x2D, 0xE9, 0xC5, 0xE8, 0x47, 0xCC, 0x32, 0x15, 0xA2, 0x98, 0xD1, 0xB9, 0x98, 0x84, 0xD6, 0x8A, 0x43, 0xA8, 0x3A, 0x8C, 0x23, 0x63, 0x47, 0x28, 0x75, 0x14, 0xA2, 0xB7, 0xAA, 0x77, 0xA7, 0x07, 0x27, 0xBF, 0x37, 0xA3, 0xBE, 0x69, 0x98, 0x2F, 0x87, 0x77, 0x8A, 0xA2, 0x8C, 0x72, 0x18, 0x32, 0xC6, 0x4B, 0xE5, 0x13, 0x96, 0x46, 0x94, 0x8E, 0xA2, 0xAF, 0x6B]

响应数据格式：

- 校验成功的响应数据：“ENDPASS@”
- 公钥校验失败的响应数据：“END\_PUBKEY\_FAIL@”
- 签名校验指令数据错误的响应数据：“END\_DATA\_FAIL@”
- 签名校验失败的响应数据：“END\_VERIFY\_FAIL@”
- 参数错误的响应数据：“END\_PARAMETER\_FAIL@”

### C.3 关闭调试指令

调试鉴权成功后可通过该指令关闭调试

请求数据格式：“DIS” + type + “@”

响应数据格式：

- 关闭成功的响应数据：“ENDPASS@”
- 关闭失败的响应数据：“END\_PARAMETER\_FAIL@”

### C.4 测试指令

该指令用于测试，例如请求数据为“TESTabcd@”，响应数据为“TESTabcd”



## 参考文献

- [1] O. IP, 《OSR Standard eHSM 技术参考手册》.
- [2] OSR, 《OSR eHSM 外部依赖 API 接口参考手册》.
- [3] OSR, 《OSR eHSM 国密二级认证技术参考手册》.